



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CAMPUS DI CESENA
DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
Corso di Laurea in Ingegneria e Scienze Informatiche

APPLICAZIONE DI TECNOLOGIE IOT PER LA REALIZZAZIONE DI SMART MUSEUM: IL CASO DI STUDIO DI VILLA SILVIA

Elaborato in
SISTEMI EMBEDDED E INTERNET-OF-THINGS

Relatore

Prof. ALESSANDRO RICCI

Presentata da

RICCARDO POLAZZI

Correlatore

FRANCO SEVERI

Anno Accademico 2024 – 2025

Indice

Introduzione	vii
1 Tecnologie smart e contesto applicativo	1
1.1 Internet of Things (IoT)	1
1.1.1 Architettura dell'IoT a cinque livelli	1
1.1.2 Esempio di funzionamento in architettura a cinque livelli	2
1.2 Smart Environment e Smart Museum	3
1.2.1 Vantaggi dei Musei Intelligenti	4
1.3 Caso di studio di Villa Silvia	4
2 Analisi del contesto e dei requisiti	7
2.1 Caratteristiche della stanza	7
2.2 Motivazioni e obiettivi generali del sistema	9
2.3 Attori coinvolti e contesto d'uso	10
2.3.1 Attori principali	10
2.3.2 Sceneggiatura funzionale della visita guidata	10
2.3.3 Diagramma dei casi d'uso	12
2.4 Requisiti funzionali	13
2.5 Requisiti non funzionali	14
2.6 Vincoli del sistema	14
2.6.1 Vincoli tecnologici	15
2.6.2 Vincoli museali	15
3 Progettazione del sistema	17
3.1 Architettura generale del sistema	17
3.1.1 Coordinatore centrale	19
3.1.2 Stazioni video	19
3.1.3 Stazione audio	21
3.1.4 Plafoniere intelligenti	22
3.2 Modalità operative del sistema	24
3.3 Progettazione della comunicazione e coordinazione	25
3.3.1 Scelte architetturali: motivazioni per la scelta di MQTT	25

3.3.2	Definizione dei topic	26
3.3.3	Sequenza di comunicazione tra dispositivi	28
4	Implementazione del sistema	29
4.1	Implementazione del coordinatore	29
4.1.1	Struttura della rete e servizio Mqtt	29
4.1.2	Applicazione di orchestrazione logica	30
4.2	Implementazione delle stazioni video	32
4.2.1	Hardware	33
4.2.2	Software	35
4.3	Implementazione della stazione audio	45
4.3.1	Hardware	45
4.3.2	Software	45
4.4	Implementazione delle plafoniere intelligenti	48
4.4.1	Hardware	48
4.4.2	Software	50
4.5	Implementazione del Modulo di acquisizione IR	52
4.5.1	Hardware	53
4.5.2	Software	54
5	Configurazione dei dispositivi	55
5.1	Configurazioni generali	55
5.1.1	Installazione del sistema operativo	55
5.1.2	Installazione di Samba	55
5.1.3	Installazione di Java e Maven	57
5.1.4	Connessione alla rete	57
5.2	Configurazione del dispositivo coordinatore	59
5.2.1	Configurazione dell'hotspot Wi-Fi	59
5.2.2	Installazione e configurazione del broker MQTT	60
5.2.3	Creazione del servizio di avvio automatico della logica di coordinamento	61
5.3	Configurazione del dispositivo video	62
5.3.1	Impedire lo standby dell'ambiente grafico	63
5.3.2	Controllo dello schermo tramite HDMI-CEC	63
5.3.3	Disabilitazione delle notifiche USB	63
5.3.4	Nascondere il cursore del mouse nell'interfaccia grafica	64
5.3.5	Avvio automatico del software videoPlayer	65
5.4	Configurazione del dispositivo audio	66
5.4.1	Avvio automatico del software audioPlayer	66
5.5	Configurazione dei moduli di trasmissione e ricezione IR	67

6	Testing del Sistema	69
6.1	Obiettivi e metodologia di testing	69
6.2	Fase 1 – Test funzionali locali	69
6.3	Fase 2 – Test dei sottosistemi in rete con MQTT	72
6.4	Fase 3 – Collaudo completo in ambiente reale	74
	Conclusioni	77

Introduzione

Negli ultimi anni, la diffusione delle tecnologie smart e dell'Internet of Things (IoT) ha reso possibile lo sviluppo di ambienti intelligenti capaci di adattarsi dinamicamente alle esigenze degli utenti e di automatizzare numerosi processi. Questa rivoluzione digitale ha interessato non solo ambiti industriali e domestici, ma anche contesti culturali e museali, nei quali le tecnologie smart offrono nuove opportunità per migliorare l'esperienza di visita e semplificare la gestione delle infrastrutture.

La presente tesi si inserisce all'interno di questo scenario, documentando la parte di analisi, progettazione, implementazione, configurazione e testing di un sistema intelligente realizzato per il Museo della Musica Meccanica di Villa Silvia, situato a Cesena. Obiettivo del lavoro è la realizzazione di una smart room multimediale in grado di orchestrare autonomamente video, audio e luci durante una visita guidata, garantendo semplicità d'uso, flessibilità e controllo centralizzato.

Lo sviluppo del progetto ha seguito un percorso strutturato, rispecchiato nell'organizzazione della tesi. Il Capitolo 1 introduce il contesto teorico e tecnologico di riferimento, illustrando i concetti fondamentali dell'Internet of Things e degli ambienti intelligenti. In particolare, viene approfondita la definizione di smart museum, analizzando come le tecnologie smart possano essere applicate al contesto museale per migliorarne la gestione e l'esperienza del visitatore.

Nel Capitolo 2 viene presentata un'analisi dettagliata del caso di studio, ovvero il Museo della Musica Meccanica di Villa Silvia. Si descrivono le caratteristiche fisiche della stanza oggetto di intervento, le esigenze funzionali, i vincoli imposti dal contesto museale e gli attori coinvolti nel sistema. In questa fase sono stati definiti gli obiettivi principali del progetto, sia dal punto di vista tecnologico che esperienziale, e sono stati individuati i requisiti funzionali e non funzionali a cui il sistema dovrà rispondere.

Il Capitolo 3 è dedicato alla fase di progettazione dell'architettura del sistema. Viene descritta in dettaglio la struttura distribuita basata su più sottosistemi, con particolare attenzione alle modalità operative e ai meccanismi di comunicazione tra i vari componenti.

Nel Capitolo 4 si entra nel vivo dell'implementazione del sistema, fornendo una descrizione approfondita dell'hardware e del software impiegati nei vari sottosistemi. Ogni componente è stato sviluppato e configurato per operare in modo autonomo ma coordinato, secondo la logica stabilita in fase progettuale.

Il Capitolo 5 documenta la configurazione dei dispositivi e l'automazione dei servizi all'avvio, garantendo il corretto funzionamento del sistema sin dalla fase di accensione. Vengono fornite istruzioni operative per la preparazione dei dispositivi, la configurazione della rete locale, l'installazione dei servizi necessari e l'avvio automatico delle applicazioni.

Infine, il Capitolo 6 descrive il processo di testing e validazione del sistema. I risultati ottenuti confermeranno l'affidabilità, la modularità e la reattività del sistema, dimostrando la sua efficacia nel contesto museale reale.

La realizzazione di questo progetto ha permesso di affrontare problematiche concrete relative all'integrazione di componenti embedded in un ambiente reale e vincolato come quello museale, mostrando come l'IoT possa essere applicato efficacemente anche in contesti culturali. La tesi si propone quindi di costituire un contributo pratico alla digitalizzazione dei musei e all'adozione di soluzioni smart per la valorizzazione del patrimonio artistico e storico.

Capitolo 1

Tecnologie smart e contesto applicativo

1.1 Internet of Things (IoT)

Negli ultimi decenni, il rapido avanzamento delle tecnologie di comunicazione, miniaturizzazione e automazione ha portato alla nascita di un nuovo paradigma informatico: l'*Internet of Things* (IoT). Con questo termine si fa riferimento a un ecosistema in cui oggetti fisici, dotati di sensori, attuatori, capacità computazionali e connettività, sono in grado di acquisire dati dall'ambiente, elaborarli localmente o tramite servizi remoti, e comunicare tra loro o con altre piattaforme digitali [2].

Il termine "Internet" richiama l'infrastruttura globale di comunicazione basata su protocolli standard (TCP/IP), mentre "Things" evidenzia il coinvolgimento diretto degli oggetti fisici nel processo informativo [4]. In questo contesto, dispositivi di uso quotidiano si trasformano in nodi intelligenti e autonomi di una rete distribuita, capaci di percepire e reagire all'ambiente circostante in tempo reale.

1.1.1 Architettura dell'IoT a cinque livelli

Per rispondere alla crescente complessità dei sistemi IoT e alla varietà di dispositivi coinvolti, è diventata comune un'architettura logica a cinque livelli. Questo modello consente di separare le funzionalità fisiche, astratte, applicative e gestionali del sistema, offrendo maggiore flessibilità e scalabilità. [1]

- **Objects Layer:** è composto da sensori e attuatori fisici che rilevano variabili ambientali (temperatura, luce, umidità, movimento, ecc.) o agiscono sul mondo reale. È il livello in cui i dati vengono generati e digitalizzati.

- **Object Abstraction Layer:** si occupa della trasmissione dei dati provenienti dal livello fisico, utilizzando tecnologie di comunicazione come Wi-Fi, ZigBee, LoRa, Bluetooth Low Energy o reti cellulari. Include anche funzioni di pre-elaborazione, filtraggio, e instradamento dei dati, oltre all'interfacciamento con servizi cloud e edge computing.
- **Service Management Layer:** funge da livello di orchestrazione tra i servizi e i dispositivi. Gestisce l'associazione tra richieste applicative e risorse fisiche, indipendentemente dalla piattaforma hardware sottostante. Implementa anche logiche decisionali di base e funzioni di middleware.
- **Application Layer:** fornisce i servizi intelligenti finali all'utente. Esempi includono dashboard, app mobili, notifiche e sistemi di controllo.
- **Business Layer:** è il livello strategico e gestionale. Analizza i dati raccolti dai livelli inferiori, produce report, indicatori di performance e supporta il processo decisionale. È responsabile dell'ottimizzazione complessiva del sistema, della privacy e della sicurezza.

Questa architettura consente di isolare le responsabilità logiche, aumentare la modularità e facilitare l'integrazione di componenti eterogenei in ambienti IoT complessi.

1.1.2 Esempio di funzionamento in architettura a cinque livelli

Per comprendere meglio questa architettura, si consideri un sistema IoT per il monitoraggio ambientale in un edificio scolastico.

- Nel **Objects Layer**, sensori installati nelle aule rilevano temperatura, umidità e concentrazione di CO₂.
- L'**Object Abstraction Layer** utilizza una rete Wi-Fi per trasmettere i dati a un gateway, che li pre-elabora e li invia a una piattaforma cloud.
- Il **Service Management Layer** interpreta i dati ricevuti, confrontandoli con soglie di riferimento. Se ad esempio la temperatura supera 26°C, viene generata un'azione per attivare il sistema di ventilazione.
- L' **Application Layer** espone una dashboard che consente agli insegnanti o al personale tecnico di visualizzare l'andamento delle condizioni ambientali, ricevere notifiche e intervenire manualmente se necessario.

- Infine, il **Business Layer** aggrega i dati raccolti per generare report settimanali sull'efficienza energetica dell'edificio, supporta decisioni strategiche, e garantisce la conformità ai requisiti di privacy e sostenibilità.

Questo esempio mostra come un'architettura multilivello consenta una gestione efficiente, modulare e scalabile dei sistemi IoT anche in contesti reali e distribuiti.

1.2 Smart Environment e Smart Museum

Gli *smart environment* sono ambienti fisici arricchiti con tecnologie progettate per percepire il contesto, elaborare informazioni in tempo reale e rispondere dinamicamente a eventi o condizioni ambientali. Tali sistemi si fondano sulle infrastrutture dell'IoT per automatizzare e ottimizzare i processi interni, con l'obiettivo di migliorare l'esperienza degli utenti e incrementare l'efficienza operativa complessiva [3].

All'interno di questi ambienti intelligenti, i dati acquisiti da sensori distribuiti vengono analizzati per regolare in modo automatico parametri come temperatura, illuminazione o qualità dell'aria, attivare dispositivi in funzione della presenza o delle preferenze degli utenti, e gestire situazioni critiche in tempo reale.

La transizione da ambienti statici a intelligenti ha prodotto un impatto significativo in molteplici settori applicativi; un ambito particolarmente promettente è quello culturale, e in particolare i cosiddetti *Smart Museum*. Con questa espressione si fa riferimento a musei che integrano tecnologie IoT per arricchire l'esperienza del visitatore, automatizzare la gestione degli spazi e valorizzare il patrimonio esposto. Le soluzioni adottate possono includere sensori per il rilevamento della presenza, dispositivi di attuazione per l'attivazione di luci o contenuti audiovisivi, schermi interattivi, piattaforme cloud per il coordinamento tra dispositivi, o ancora sistemi in grado di inviare contenuti personalizzati su smartphone e tablet dei visitatori.

L'obiettivo di questi sistemi non è solo informare, ma coinvolgere l'utente in un'esperienza culturale più profonda, dinamica e contestuale. L'interazione può avvenire sia attraverso installazioni fisiche (come totem multimediali, proiezioni e pannelli interattivi), sia tramite contenuti multilingua o personalizzati ricevuti sui propri dispositivi mobili in base alla posizione, al comportamento o agli interessi specifici [5].

Inoltre, l'adozione di tecnologie IoT consente anche la raccolta di dati utili per l'analisi dei flussi, la pianificazione degli spazi, la regolazione automatica dell'ambiente espositivo e l'ottimizzazione gestionale nel lungo termine.

1.2.1 Vantaggi dei Musei Intelligenti

Tra i principali benefici offerti dai musei intelligenti si possono evidenziare:

- **Esperienza immersiva:** l'utilizzo di contenuti multimediali, dispositivi interattivi, luci dinamiche o media personalizzati contribuisce a coinvolgere il visitatore in modo più profondo e sensoriale.
- **Monitoraggio ambientale:** sensori dedicati rilevano parametri come temperatura, umidità e illuminazione, garantendo condizioni ottimali per la conservazione delle opere d'arte.
- **Accessibilità aumentata:** l'integrazione di tecnologie assistive supporta la fruizione museale da parte di persone con disabilità, migliorando l'inclusività dell'esperienza.
- **Ottimizzazione gestionale:** i dati raccolti possono essere analizzati per supportare decisioni strategiche in ambiti come logistica, promozione, manutenzione o layout espositivo.

In conclusione, i musei intelligenti rappresentano una naturale evoluzione del concetto di ambiente interattivo e connesso, in grado di valorizzare il patrimonio culturale attraverso l'integrazione tra percezione del contesto, automazione, analisi dei dati e personalizzazione dell'esperienza.

1.3 Caso di studio di Villa Silvia

Villa Silvia-Carducci è una raffinata dimora ottocentesca situata sulle colline di Cesena, immersa in un paesaggio tranquillo e suggestivo. In passato è appartenuta alla contessa Silvia Pasolini Zanelli, figura di spicco dell'aristocrazia romagnola e donna di grande cultura. Sotto la sua guida, la villa divenne un importante punto di riferimento per il mondo intellettuale del tempo, trasformandosi in un salotto letterario frequentato da poeti, artisti e pensatori.

Tra gli ospiti più noti vi fu il poeta Giosuè Carducci, che trovava nella villa un luogo di pace e ispirazione. Il legame tra il poeta e la contessa fu profondo e duraturo, tanto che Carducci vi soggiornò spesso, lasciando tracce significative del suo passaggio in lettere, scritti e ricordi. Questo rapporto ha contribuito a rendere Villa Silvia un luogo simbolico, dove cultura, memoria e bellezza si fondono.

Nel corso degli anni, la villa ha vissuto diverse trasformazioni, ma ha sempre mantenuto la sua vocazione culturale. Oggi è sede del **Museo della Musica Meccanica**, una realtà unica in Italia che ospita una ricchissima collezione di

strumenti musicali automatici, tra cui carillon, organetti, orchestrioni, pianole, grammofoni e molti altri dispositivi.

Accanto all'attività museale, Villa Silvia è anche sede del **Giardino Parlante**, un'iniziativa multimediale pensata per valorizzare il parco della villa attraverso percorsi sonori, narrazioni interattive e pannelli informativi. I visitatori possono accedere ai contenuti tramite stazioni installate lungo il percorso, immergendosi in racconti legati alla storia della villa, dei personaggi che la hanno frequentata e degli strumenti musicali conservati nel museo. Il Giardino Parlante rappresenta un primo esempio di integrazione tra patrimonio culturale e tecnologie digitali.

Tuttavia, le tecnologie attualmente in uso risultano perlopiù statiche e non interconnesse. Da qui nasce l'interesse per l'introduzione di soluzioni basate sull'Internet of Things (IoT), con l'obiettivo di valorizzare ulteriormente gli spazi della villa e potenziare l'interazione tra visitatore e contenuto in modo dinamico. Questo approccio rappresenta un passo significativo verso la trasformazione di Villa Silvia in un ambiente culturale intelligente e coinvolgente.



Figura 1.1: Vista esterna di Villa Silvia

Capitolo 2

Analisi del contesto e dei requisiti

Nel contesto del Museo della Musica Meccanica di Villa Silvia, è stato avviato un progetto di innovazione tecnologica volto a trasformare una delle sale espositive, denominata "stanza Bonci", in un ambiente intelligente in grado di supportare e automatizzare le visite guidate.

La stanza Bonci è utilizzata per illustrare la storia e il funzionamento degli strumenti musicali meccanici esposti, attraverso il racconto diretto da parte delle guide. Si tratta di una sala con caratteristiche storiche e architettoniche di rilievo, attualmente priva di stazioni multimediali o dispositivi digitali.

Prima di definire le strategie progettuali e le soluzioni tecnologiche più adatte, è fondamentale analizzare in modo approfondito il contesto di partenza. In questo capitolo si descrivono le condizioni attuali dell'ambiente museale, le dinamiche operative della visita e gli attori coinvolti, al fine di individuare con precisione i requisiti funzionali e non funzionali del sistema da realizzare. Verranno inoltre evidenziati i vincoli architettonici e gestionali che influenzeranno la progettazione, e presentati i possibili scenari d'uso del sistema futuro, che saranno sviluppati nei capitoli successivi.

2.1 Caratteristiche della stanza

La stanza Bonci, oggetto dell'intervento progettuale, è uno degli ambienti più significativi del Museo della Musica Meccanica di Villa Silvia. La stanza prende il nome da Alessandro Bonci, celebre tenore cesenate, e rappresenta un momento centrale all'interno del percorso museale, dedicato all'illustrazione delle trasformazioni della musica automatica tra XIX e XX secolo. La stanza oggetto dell'intervento ha una forma pressoché rettangolare, con una superficie di circa 50 m². L'ambiente è caratterizzato da soffitti alti e decorati, che con-

tribuiscono al valore storico e architettonico del museo. All'interno della sala sono presenti numerosi strumenti musicali meccanici distribuiti lungo le pareti, che costituiscono parte integrante dell'allestimento museale. L'illuminazione artificiale è attualmente garantita da sei plafoniere LED installate a soffitto, equamente distribuite in modo da coprire l'intera superficie dell'ambiente. Sul lato opposto rispetto all'ingresso è collocato un piccolo palco, attualmente utilizzato per presentazioni e attività didattiche. Di fronte al palco è già presente un videoproiettore installato, mentre in un angolo adiacente si trova una cassa audio, utilizzata per la riproduzione di contenuti sonori durante gli eventi.



Figura 2.1: Vista interna della stanza Bonci.

2.2 Motivazioni e obiettivi generali del sistema

L'analisi dell'ambiente museale ha evidenziato l'assenza di strumenti tecnologici avanzati per il supporto delle visite guidate. Nella configurazione attuale, l'interazione con i visitatori si basa esclusivamente sulla narrazione della guida, senza l'ausilio di contenuti multimediali, automatismi o adattamenti ambientali. Sebbene questo approccio mantenga un forte valore umano e culturale, può risultare limitante in termini di immersività ed efficacia comunicativa, specialmente in un contesto museale contemporaneo.

Alla luce di ciò, il progetto si propone di evolvere la stanza Bonci in un ambiente intelligente e reattivo, capace di integrare tecnologie embedded e sistemi IoT per migliorare l'esperienza di visita senza snaturare il ruolo centrale della guida.

Gli obiettivi principali che guidano questa trasformazione sono:

- **Introdurre postazioni multimediali interattive:** dovranno essere installati schermi dedicati alla riproduzione di video tematici, capaci di arricchire la narrazione della guida con contenuti visivi e sonori di alta qualità. Ogni video dovrà essere associato a una specifica tappa del percorso e integrato con piccole luci locali, dedicate alla singola stazione. Queste luci dovranno attivarsi automaticamente in momenti precisi del video, enfatizzando elementi espositivi menzionati nella narrazione.
- **Permettere alla guida di scandire il ritmo della visita in modo armonioso e invisibile:** la guida dovrà mantenere il pieno controllo sul ritmo e sulla sequenza della visita, potendo attivare i contenuti video delle diverse stazioni nel momento che ritiene più opportuno. Questa attivazione dovrà avvenire attraverso modalità discrete e non invasive, così da preservare la continuità narrativa e garantire un'esperienza fluida e naturale per i visitatori.
- **Adattare luci e audio allo stato della visita:** le plafoniere e il sistema audio dovranno essere gestiti in modo automatico, adattandosi dinamicamente allo stato della visita e alla posizione del gruppo nella stanza. Questa sincronizzazione dovrà contribuire a creare un'atmosfera coerente e immersiva in ogni fase del percorso. Al contempo, dovrà alleggerirsi il carico operativo sulla guida, che potrà concentrarsi esclusivamente sulla narrazione.

2.3 Attori coinvolti e contesto d'uso

Per comprendere al meglio il funzionamento del sistema proposto e il suo impatto sul contesto museale, è necessario analizzare gli attori che interagiranno direttamente o indirettamente con la smart room e definire il contesto d'uso operativo.

2.3.1 Attori principali

- **Guida museale:** rappresenta l'utente principale del sistema. La guida conduce le visite, interagisce direttamente con l'ambiente e scandisce il ritmo dell'esperienza. Inoltre, è responsabile dell'attivazione e disattivazione della smart room, avviandola all'inizio della visita e portandola in modalità standby al termine. Non deve essere richiesto alcuna capacità tecnica: ogni automazione deve essere trasparente.
- **Visitatori:** Assistono ai contenuti multimediali e agli effetti ambientali (luci, suoni). Interagiscono passivamente con il sistema attraverso la loro presenza nello spazio.
- **Personale tecnico:** si occupa della manutenzione, aggiornamento e configurazione del sistema. È responsabile della gestione e dell'aggiornamento dei contenuti multimediali (video e audio) e della regolazione dei parametri di sistema (come il tempismo di attivazione delle luci locali).

2.3.2 Sceneggiatura funzionale della visita guidata

Per comprendere appieno l'obiettivo finale richiesto dal proprietario del museo Franco Severi, in questa sezione viene illustrato il flusso ideale della visita guidata all'interno della stanza Bonci.

1. **Ingresso nella stanza:** la guida e i visitatori entrano nella stanza. In questa fase, le plafoniere LED e l'impianto audio ambientale sono attivi al livello massimo preimpostato. Questo assetto garantisce un ambiente accogliente e ben illuminato, con un piacevole sottofondo sonoro non invasivo. Tra tutte le postazioni video presenti, solo la prima (relativa alla prima tappa della visita) mostra un'immagine fissa, selezionata per attirare l'attenzione dei visitatori; tutti gli altri schermi appaiono spenti, così da non distrarre il pubblico e mantenere la scena ordinata.
2. **Introduzione della guida presso la prima stazione:** la guida si posiziona nei pressi della prima postazione e introduce brevemente la

funzione storica della stanza e la natura degli strumenti esposti. L'attenzione dei visitatori è naturalmente guidata verso la stazione attiva, grazie alla presenza dell'immagine fissa sul display.

3. **Attivazione della prima postazione video:** l'attivazione della prima stazione segna l'inizio effettivo dell'esperienza immersiva. Quando la guida avvia il contenuto, si innesca una sequenza coordinata pensata per catturare l'attenzione e creare atmosfera:
 - Le plafoniere LED e l'audio ambientale vengono automaticamente abbassati, trasformando l'ambiente da spazio espositivo neutro a scenario narrativo immersivo.
 - Il video della postazione viene avviato, con il proprio audio riprodotto localmente tramite il display.
4. **Proseguimento della visita:** al termine del video di una postazione, la stazione stessa si disattiva automaticamente: lo schermo si spegne o mostra una schermata nera. Contemporaneamente, la postazione successiva si "accende", mostrando un'immagine statica per guidare l'attenzione dei visitatori verso il nuovo punto focale. A questo punto, il sistema resta in attesa dell'attivazione da parte della guida, che darà il via alla riproduzione del nuovo video. L'avvio del video attiva automaticamente le luci locali associate agli oggetti citati nella narrazione, che si accendono e si spengono in momenti specifici per enfatizzare i punti salienti del contenuto.
5. **Chiusura della visita:** una volta ripetuto il processo descritto per tutte le postazioni, e completata la riproduzione dell'ultimo contenuto video, tutte le stazioni risultano spente e il sistema ripristina automaticamente il livello massimo di illuminazione ambientale e di volume dell'audio di sottofondo. Questo cambiamento segnala in modo implicito e naturale la conclusione della visita all'interno della stanza Bonci, accompagnando il gruppo verso la fase di uscita.

L'intero processo di visita viene percepito in modo naturale dai visitatori, mantenendo un alto grado di immersività. La guida conserva il controllo sul ritmo della visita, ma le interazioni con il sistema avvengono in modo discreto, evitando di interrompere il flusso narrativo. Le luci locali e le sincronizzazioni audio-video conferiscono un forte impatto scenografico, migliorando l'efficacia comunicativa della narrazione.

2.3.3 Diagramma dei casi d'uso

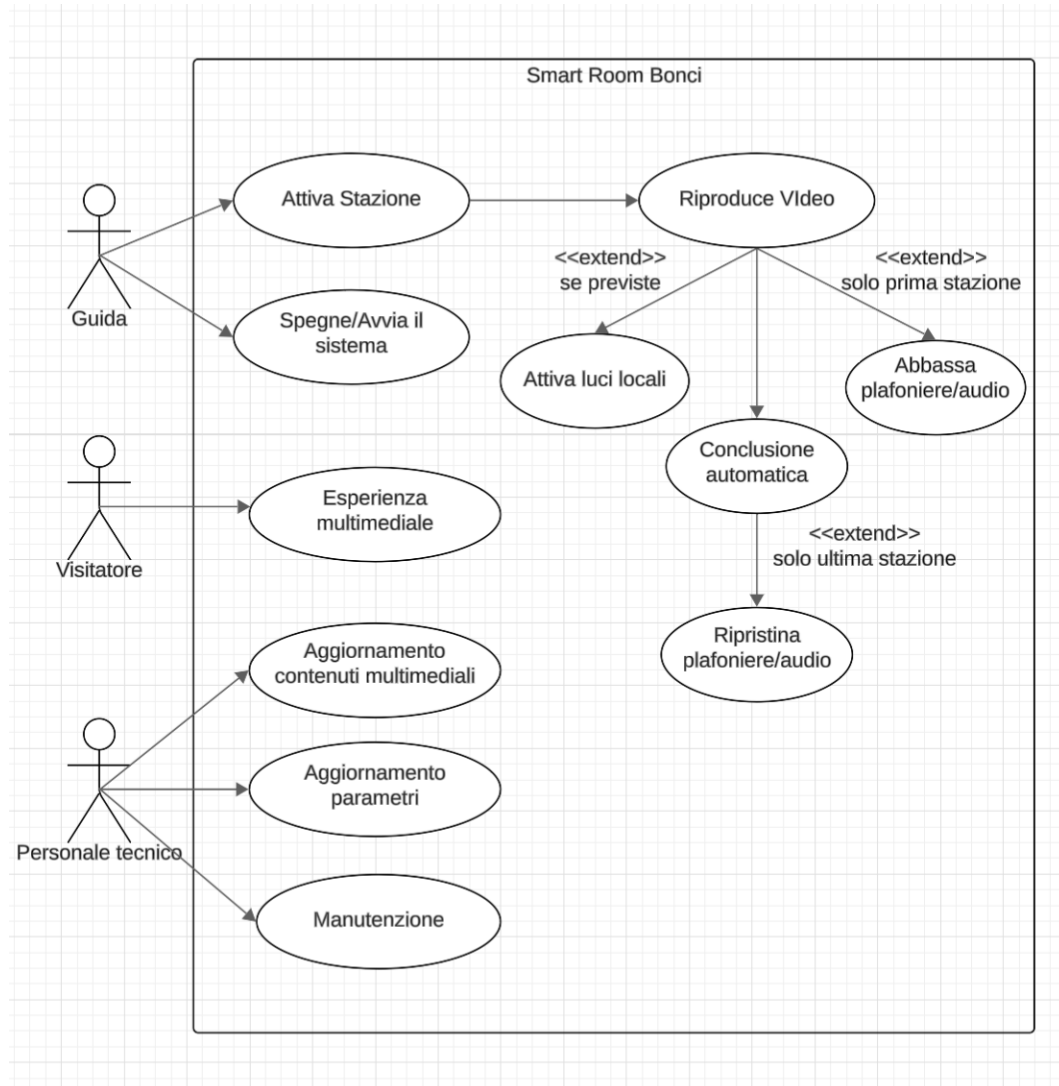


Figura 2.2: Diagramma dei casi d'uso della smart room

2.4 Requisiti funzionali

I requisiti funzionali descrivono il comportamento atteso del sistema all'interno della stanza Bonci durante le visite guidate. Sono stati individuati a partire dall'analisi del contesto d'uso, del flusso della visita e delle esigenze operative espresse dai responsabili museali.

- **RF1 – Attivazione sequenziale delle postazioni video:** Il sistema deve consentire l'attivazione delle postazioni video in modo sequenziale, in base alla posizione della guida. Ogni attivazione comporta l'avvio del contenuto, l'accensione delle luci locali e l'adattamento dell'ambiente (audio e luce).
- **RF2 – Rilevamento automatico della presenza:** Le postazioni devono rilevare automaticamente la presenza della guida e attivarsi senza bisogno di interazioni manuali.
- **RF3 – Gestione automatica e sincronizzata di illuminazione e audio ambientale:** Il sistema deve regolare automaticamente l'intensità delle plafoniere e il volume dell'audio ambientale. La gestione deve essere coerente con l'andamento della visita.
- **RF4 – Acquisizione automatica dei contenuti multimediali:** Le postazioni video e audio devono rilevare automaticamente dispositivi USB contenenti nuovi contenuti multimediali (es. video, audio) e renderli disponibili per la riproduzione, senza interventi tecnici.
- **RF5 – Configurabilità dei parametri operativi:** Il sistema deve consentire al personale tecnico di modificare, senza ricorrere al codice, parametri come:
 - i video o gli audio che vengono riprodotti dalle stazioni;
 - il ritardo tra inizio video e accensione/spegnimento delle luci locali;
- **RF6 – Modalità di risparmio energetico automatica:** Durante i periodi di inattività (es. chiusura del museo), il sistema deve disattivare temporaneamente i componenti non essenziali (schermi, video, audio), mantenendosi in ascolto per eventuali riattivazioni.

2.5 Requisiti non funzionali

I requisiti non funzionali descrivono le qualità generali che il sistema deve garantire per offrire un'esperienza efficace, sostenibile e rispettosa del contesto museale.

- **RNF1 – Usabilità:** Il sistema deve essere utilizzabile da personale non tecnico. Le attivazioni devono avvenire in modo automatico o tramite azioni semplici, evitando interfacce complesse.
- **RNF2 – Affidabilità:** Tutti i dispositivi devono garantire funzionamento stabile e continuo durante la visita, senza malfunzionamenti o interruzioni.
- **RNF3 – Manutenibilità:** Il sistema deve essere progettato in modo modulare per facilitare l'ispezione, l'aggiornamento e la sostituzione delle singole componenti hardware o software.
- **RNF4 – Prestazioni:** Le risposte agli eventi (es. trigger della guida) devono avvenire in tempo reale, senza ritardi percepibili o asincronie tra i dispositivi.
- **RNF5 – Scalabilità:** L'architettura del sistema deve consentire l'aggiunta di nuove postazioni o l'estensione ad altri ambienti museali senza dover rivedere radicalmente l'impianto esistente.
- **RNF7 – Resilienza a interruzioni elettriche:** In caso di blackout, il sistema deve poter riprendere il funzionamento automaticamente, mantenendo coerenza rispetto allo stato precedente.
- **RNF8 – Efficienza energetica:** L'intero sistema deve garantire bassi consumi sia durante l'uso attivo che in modalità standby.

2.6 Vincoli del sistema

Durante la fase iniziale di analisi della stanza Bonci, sono emersi alcuni vincoli rilevanti, legati sia al contesto architettonico sia alle caratteristiche tecniche e infrastrutturali dell'ambiente. Tali vincoli dovranno essere attentamente considerati nella progettazione e realizzazione della smart room all'interno della stanza, in quanto influenzano direttamente le scelte relative all'hardware, al software, alle modalità di installazione e alla gestione operativa del sistema.

2.6.1 Vincoli tecnologici

Dal punto di vista tecnico, il sistema è soggetto ai seguenti vincoli:

- **Compatibilità con dispositivi non smart:** nella stanza Bonci sono state installate plafoniere non dotate di funzionalità di controllo remoto, scelte per motivi estetici e di illuminazione. Questo vincolo impone la necessità di integrare nel sistema domotico dispositivi di illuminazione non smart, senza modificarne l'aspetto né la struttura.
- **Assenza di connessione Internet:** la stanza non è servita da una rete Internet stabile o permanente. Il sistema, pertanto, dovrà operare in modo completamente autonomo tramite infrastruttura locale, senza dipendere da servizi esterni o accessi alla rete.
- **Sicurezza elettrica:** tutti i dispositivi installati devono essere conformi alle normative vigenti in materia di sicurezza elettrica. È richiesto l'uso di basse tensioni e componenti certificati CE, idonei all'utilizzo in ambienti aperti al pubblico.

2.6.2 Vincoli museali

Essendo la stanza collocata in una villa storica, è necessario rispettare precisi criteri di tutela e conservazione:

- **Non invasività:** non è consentito praticare fori permanenti, danneggiare superfici originali o modificare strutturalmente pareti, soffitti o pavimentazioni.
- **Compatibilità ambientale:** i dispositivi installati non devono alterare le condizioni termoigrometriche della stanza o interferire con la conservazione degli strumenti meccanici esposti.
- **Integrazione estetica:** tutti i componenti tecnologici (sensori, dispositivi, cablaggi, schermi) devono essere installati in modo da risultare il più possibile invisibili al pubblico, rispettando l'ordine visivo e l'armonia dell'allestimento esistente. È fondamentale evitare qualsiasi elemento che possa compromettere l'esperienza estetica del visitatore o l'identità storica della stanza.

Il rispetto di questi vincoli è fondamentale per garantire la sostenibilità del progetto nel lungo termine, la compatibilità con l'ambiente museale e l'accettazione da parte degli operatori culturali e tecnici del museo.

Capitolo 3

Progettazione del sistema

La progettazione si è basata sull'idea di costruire un sistema distribuito, dinamico e adattabile, in grado di coordinare in modo efficace sensori, attuatori e contenuti multimediali, garantendo allo stesso tempo la massima compatibilità con l'ambiente museale e con le esigenze del personale non tecnico.

Un principio fondamentale dell'architettura proposta è la modularità, intesa come capacità di suddividere il sistema in componenti indipendenti e riutilizzabili, ciascuno con responsabilità ben definite. Questa caratteristica consente una più agevole manutenzione, aggiornamento e sostituzione dei singoli elementi senza compromettere l'intero sistema.

Parallelamente, è stata perseguita la scalabilità, ovvero la possibilità di estendere facilmente il sistema nel tempo. Grazie alla sua struttura distribuita, il sistema è progettato per permettere:

- l'aggiunta di nuove postazioni o dispositivi (es. ulteriori stazioni video, nuove luci locali, schermi o sensori) con una configurazione minima;
- il riutilizzo dell'intera architettura in altre stanze del museo, replicando o adattando il sistema a contesti simili con minime modifiche.

Questi aspetti garantiscono che il sistema non solo soddisfi i requisiti attuali della stanza Bonci, ma costituisca anche una base solida per future evoluzioni ed estensioni all'interno del museo.

3.1 Architettura generale del sistema

L'architettura del sistema si fonda su una rete locale dedicata (LAN/Wi-Fi), che connette tra loro i vari sottosistemi fisici. La comunicazione tra questi avviene attraverso il protocollo MQTT, scelto per la sua leggerezza, efficienza e adeguatezza ai contesti distribuiti. Ogni sottosistema è progettato per svolgere

un ruolo specifico all'interno dell'ambiente museale intelligente, contribuendo alla creazione di un'esperienza interattiva e immersiva per i visitatori.

Il sistema si compone delle seguenti entità principali:

- **Coordinatore centrale** (1 unità)
- **Stazioni video** (4 unità)
- **Stazione audio** (1 unità)
- **Plafoniere smart** (6 unità)
- **Modulo di acquisizione IR** (1 unità, solo in fase iniziale)

Nel diagramma UML delle componenti riportato di seguito è rappresentata la struttura generale del sistema, con l'obiettivo di evidenziare le principali entità e le relazioni tra di esse. Ciascuna componente mostrata sarà analizzata nei paragrafi successivi, approfondendone il ruolo, il funzionamento e l'integrazione con le altre componenti del sistema.

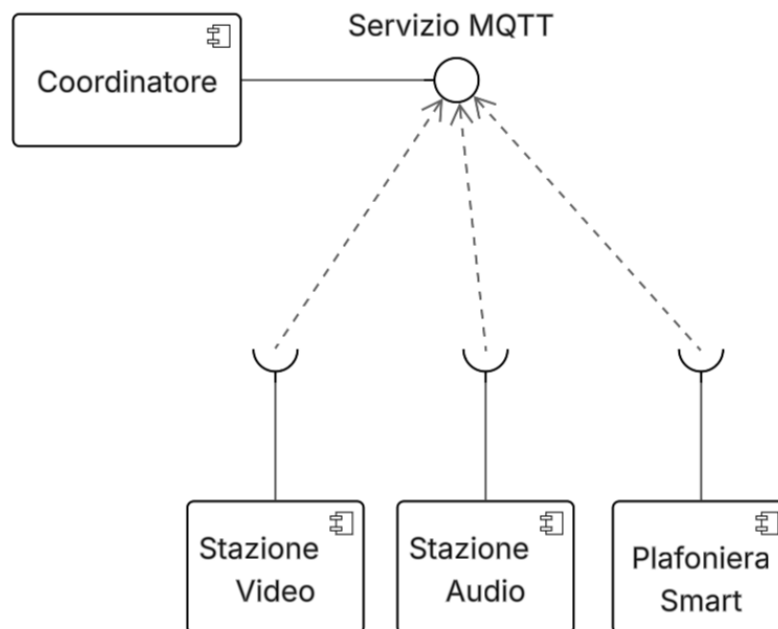


Figura 3.1: Diagramma UML delle componenti principali del sistema

3.1.1 Coordinatore centrale

Per garantire una gestione fluida e sincronizzata di tutte le componenti distribuite nel sistema museale, è previsto un nodo centrale, definito come coordinatore. Questo elemento ha il compito di orchestrare le azioni dei dispositivi periferici, assicurando che eventi come la riproduzione di video, la variazione dell'illuminazione o la diffusione audio avvengano in maniera coerente, ordinata e sincronizzata.

Il coordinatore sarà ospitato su un'unità computazionale a basso consumo, come un Raspberry Pi, in grado di fornire connettività locale, elaborazione degli eventi e coordinamento centralizzato delle operazioni.

Funzionalità principali

Il coordinatore centrale è progettato per svolgere tre funzioni principali:

1. **Punto di accesso alla rete locale (hotspot):** Il coordinatore fornisce connettività locale ai dispositivi tramite una rete Wi-Fi dedicata, garantendo una comunicazione stabile e indipendente da infrastrutture esterne.
2. **Hosting del broker MQTT:** Il coordinatore ospita localmente il broker MQTT, responsabile della comunicazione tra i diversi sottosistemi. Grazie all'architettura basata sul paradigma publish/subscribe, la trasmissione dei messaggi avviene in modo efficiente, disaccoppiato e asincrono, ideale per un sistema distribuito come quello museale.
3. **Esecuzione delle logiche di coordinamento:** Oltre a fornire il servizio di comunicazione basato su protocollo MQTT, il coordinatore ospita anche l'applicazione centrale responsabile del coordinamento del sistema. Questa applicazione, eseguita sullo stesso dispositivo del broker, rimane costantemente in ascolto dei messaggi pubblicati sui topic MQTT e, in base al loro contenuto, elabora la logica del comportamento del sistema, pubblicando a sua volta i comandi necessari per sincronizzare l'azione dei vari dispositivi coinvolti nella visita.

3.1.2 Stazioni video

Le stazioni video rappresentano uno degli elementi centrali della narrazione multimediale pensata per il percorso museale. La loro funzione principale è quella di trasmettere contenuti audiovisivi in momenti specifici della visita, in stretta relazione con il contesto artistico circostante. Ogni stazione è

concepita come un'unità modulare, progettata per essere autonoma, reattiva e perfettamente integrata nel sistema intelligente.

Ciascuna stazione video è composta da:

- **Uno schermo:** utilizzato per la riproduzione dei contenuti video, costituisce il principale canale visivo della narrazione multimediale.
- **Un Raspberry Pi:** si occupa della gestione locale del video, della ricezione dei comandi via rete e della comunicazione con il coordinatore centrale tramite protocollo MQTT. Questo dispositivo permette a ciascuna stazione di reagire in modo autonomo e coordinato agli stimoli ricevuti.
- **Un sensore di prossimità a ultrasuoni (sonar):** integrato per consentire un'attivazione discreta e automatica da parte della guida. Il rilevamento della presenza in prossimità della stazione permette l'avvio del contenuto senza necessità di interazione manuale, garantendo fluidità e coerenza narrativa.
- **Uno o più dispositivi Shelly Plug IT per l'illuminazione locale:** utilizzati per accendere o spegnere in modo sincronizzato luci locali dedicate alla stazione stessa. Le Shelly Plug IT sono prese intelligenti compatibili con il formato delle comuni prese italiane, che agiscono come intermediari tra la presa a muro e il dispositivo da alimentare (es. una lampada o una striscia LED). Inserendo la spina della luce nella smart plug e quest'ultima nella presa, è possibile controllare da remoto il passaggio di corrente, abilitando o disabilitando l'alimentazione elettrica verso il carico collegato. La gestione avviene tramite rete Wi-Fi e protocollo MQTT, rendendo questi dispositivi perfettamente integrabili nel sistema smart.

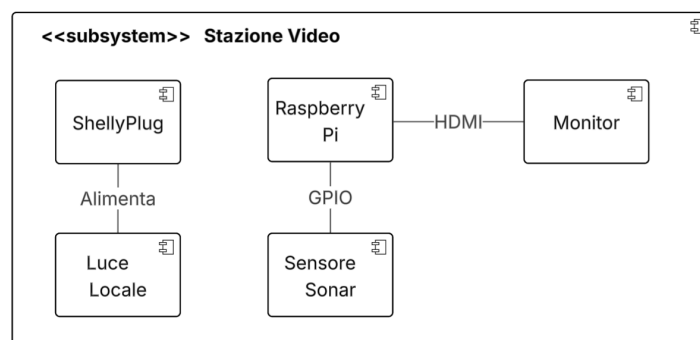


Figura 3.2: Diagramma UML della stazione video

3.1.3 Stazione audio

Parallelamente alle postazioni video, è prevista una stazione dedicata alla diffusione della narrazione di sfondo, pensata per accompagnare l'intero percorso espositivo o momenti specifici di esso. Dal punto di vista architeturale, la stazione audio è composta da:

- **Uno stereo amplificato:** collocato in un angolo della stanza, si occupa della diffusione del sottofondo sonoro ambientale, fornendo una narrazione audio coerente e armonizzata con il resto dell'esperienza multimediale, senza sovrapporsi alle altre fonti sonore presenti.
- **Un Raspberry Pi:** gestisce localmente la riproduzione della traccia audio, controlla la logica di attivazione e regolazione del volume e gestisce la comunicazione con il coordinatore centrale tramite protocollo MQTT. Ciò consente alla stazione di reagire in modo dinamico agli eventi del sistema, adattandosi al contesto della visita.

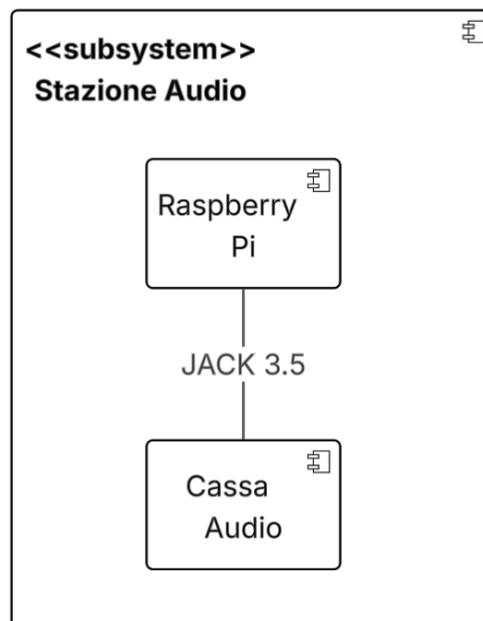


Figura 3.3: Diagramma UML della stazione audio

3.1.4 Plafoniere intelligenti

L'illuminazione generale della stanza Bonci, è affidata a sei plafoniere LED già installate. Questi dispositivi, adeguati dal punto di vista estetico e funzionale, non sono però dotati nativamente di interfacce digitali per il controllo remoto. È quindi necessario predisporre una soluzione progettuale non invasiva, in grado di integrarle nel sistema smart senza modificarne la struttura fisica né comprometterne l'aspetto estetico.

Dal punto di vista architettuale, il sistema di controllo intelligente delle plafoniere si basa sui seguenti elementi:

- **Plafoniere LED esistenti:** dispositivi già installati, compatibili con un telecomando a infrarossi fornito dal produttore. Permettono funzioni base come accensione, spegnimento e regolazione dell'intensità luminosa tramite comandi IR. Non dispongono di connettività nativa o capacità smart.
- **Modulo di trasmissione IR:** ogni plafoniera sarà abbinata a un modulo composto da un microcontrollore ESP32 connesso alla rete locale e da un trasmettitore a infrarossi. Questo modulo riceverà i comandi dal sistema centrale e li convertirà nei corrispondenti segnali IR, simulando l'uso del telecomando originale. In questo modo, si abilita un controllo remoto completo della plafoniera, mantenendo la compatibilità con il suo funzionamento originale.
- **Modulo di acquisizione IR (strumento di configurazione):** utilizzato esclusivamente nella fase iniziale, questo modulo è dotato di un microcontrollore ESP32 in collegamento con un ricevitore IR e consente di acquisire e decodificare i segnali inviati dal telecomando delle plafoniere. La sua funzione è quella di mappare i codici associati ai principali comandi (accensione, spegnimento, dimmer) per poterli poi replicare fedelmente tramite i moduli di trasmissione. Non è previsto che questo modulo faccia parte del sistema operativo, ma solo della fase di configurazione.

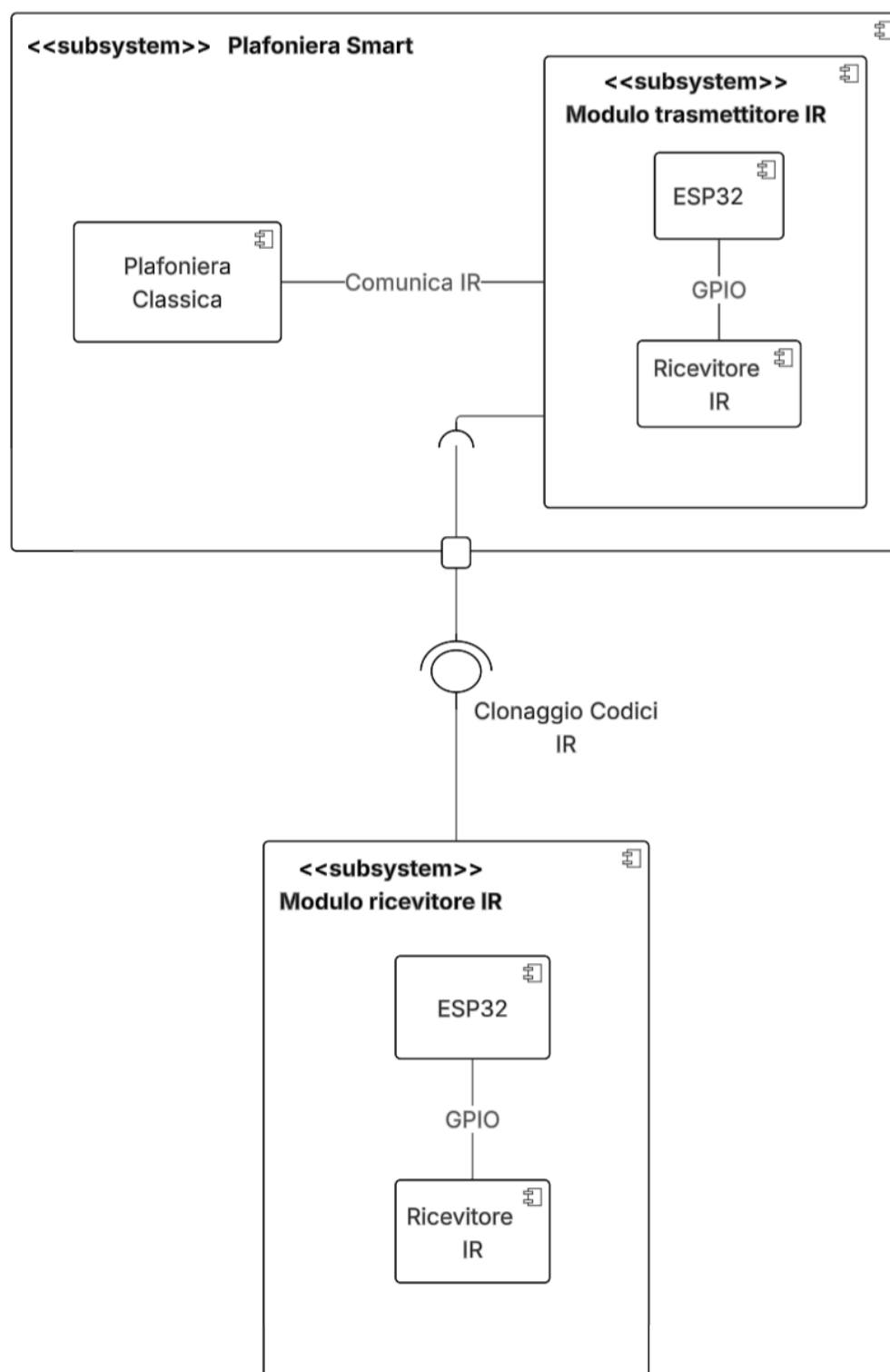


Figura 3.4: Diagramma UML della plafoniera smart

3.2 Modalità operative del sistema

Per garantire il corretto funzionamento del sistema multimediale della stanza Bonci e soddisfare i requisiti di efficienza energetica, sono state definite due modalità operative principali: modalità attiva e modalità standby. Queste modalità determinano il comportamento di tutti i dispositivi coinvolti e sono pensate per adattarsi dinamicamente alla presenza o assenza di visite guidate.

- **Modalità attiva**

È la modalità operativa standard durante lo svolgimento di una visita. In questo stato:

- tutti i dispositivi svolgono le loro funzionalità principali;
- ogni stazione video e audio esegue i propri servizi, comunica via MQTT con il coordinatore centrale e interagisce con gli altri sottosistemi;
- le plafoniere sono gestite dinamicamente in base allo stato della visita, e i microcontrollori al loro interno sono attivi;
- il coordinatore centrale mantiene la rete Wi-Fi attiva e gestisce il broker MQTT per la distribuzione dei comandi e degli stati.

- **Modalità standby**

Questa modalità è pensata per i periodi di inattività prolungata, come la notte o le ore in cui non sono previste visite. Ha lo scopo di ridurre al minimo il consumo energetico dell'intero sistema, senza però compromettere la possibilità di riattivazione remota. Per questo motivo, i dispositivi non possono essere spenti completamente, poiché uno spegnimento fisico renderebbe impossibile ricevere ulteriori comandi via rete, costringendo a un riavvio manuale tramite disconnessione e riconnessione dell'alimentazione.

In modalità standby:

- i Raspberry Pi delle stazioni video disattivano l'ambiente grafico, limitano l'uso della CPU, disattivano i servizi multimediali, disattivano i sensori di prossimità e soprattutto spengono gli schermi;
- i Raspberry Pi delle stazioni audio limitano anch'essi le loro funzioni di calcolo e disattivano la traccia audio.
- i dispositivi ESP disattivano le plafoniere e si pongono in uno stato di basso consumo;

- ogni dispositivo rimane comunque connesso alla rete e in ascolto sul proprio topic MQTT, pronto a ricevere un messaggio di riattivazione;
- solo il coordinatore centrale resta completamente operativo: mantiene la rete locale attiva, gestisce il broker MQTT e monitora lo stato del sistema.

Questa logica consente di ottenere un equilibrio tra efficienza energetica e disponibilità operativa continua: i dispositivi riducono le attività non essenziali, ma restano “in ascolto” per poter essere riattivati in qualunque momento, senza intervento fisico.

3.3 Progettazione della comunicazione e coordinazione

Uno degli elementi centrali per il corretto funzionamento della smart room è la definizione di un meccanismo di comunicazione efficace tra i diversi sottosistemi. La natura distribuita del sistema richiede una soluzione che garantisca interoperabilità, sincronizzazione e semplicità di gestione.

Per soddisfare queste esigenze, è stata progettata un’architettura di rete locale basata su Wi-Fi, in cui i dispositivi dialogano attraverso un protocollo di messaggistica efficiente. La scelta di tale protocollo, insieme alla motivazione tecnica sottostante, viene approfondita nel paragrafo seguente.

3.3.1 Scelte architetturali: motivazioni per la scelta di MQTT

Nel progettare l’architettura del sistema di comunicazione, si è valutata l’opportunità di adottare diversi protocolli per lo scambio di messaggi tra i dispositivi connessi alla rete locale. Tra le alternative considerate, il protocollo MQTT si è rivelato la soluzione più adatta, in particolare se confrontato con l’approccio basato su HTTP.

MQTT (Message Queuing Telemetry Transport) è stato progettato per ambienti distribuiti con dispositivi embedded a basse risorse, e si basa su un modello di comunicazione asincrono di tipo *publish/subscribe*. Questo approccio si è rivelato particolarmente adatto nel contesto della smart room per i seguenti motivi:

- **Leggerezza e semplicità:** MQTT è un protocollo estremamente leggero, con un overhead minimo sia in termini di larghezza di banda che

di elaborazione, ideale per dispositivi a risorse limitate come ESP32 o microcontrollori.

- **Comunicazione asincrona e disaccoppiata:** grazie al paradigma *publish/subscribe*, i dispositivi non devono conoscere l'indirizzo degli altri nodi; possono semplicemente pubblicare o ricevere messaggi su uno o più topic, rendendo il sistema più modulare e manutenibile.
- **Indipendenza dagli indirizzi IP:** l'architettura MQTT non richiede l'uso di IP statici o DNS locali, evitando criticità tipiche della comunicazione HTTP, che si basa su chiamate dirette tra client e server.
- **Scalabilità e flessibilità:** l'aggiunta di nuove postazioni può avvenire senza modificare la logica esistente, semplicemente definendo nuovi topic o sottoscrivendosi a quelli già esistenti.
- **Maggiore reattività:** MQTT è stato progettato per funzionare anche su reti instabili o con alta latenza, garantendo tempi di risposta migliori rispetto a HTTP in contesti embedded.

A seguito di queste considerazioni, MQTT è stato adottato come protocollo di riferimento per l'intera comunicazione del sistema, garantendo affidabilità, modularità e semplicità di gestione in uno scenario in cui è cruciale mantenere bassi i consumi e alta la reattività.

3.3.2 Definizione dei topic

Struttura dei topic

La struttura dei topic MQTT adottata è semantica, e segue la convenzione:

`stanza/componente/azione`

Dove:

- **stanza** rappresenta il prefisso relativo alla stanza, in questo caso Bonci. Questa scelta consente, in una prospettiva di scalabilità futura, di distinguere facilmente i topic associati a ciascun ambiente qualora il sistema venga esteso ad altre stanze del museo.
- **componente** è il tipo di dispositivo coinvolto (es. `videoPlayer1`, `audioPlayer`, `plafoniera6`);
- **azione** rappresenta il comando o l'evento specifico (es. `play`, `triggered`, `data`).

Elenco dei topic

Ogni dispositivo si iscrive esclusivamente ai topic rilevanti per il proprio ruolo, come riportato di seguito.

- `bonci/*/data`: inviato da ciascun dispositivo al momento della connessione al broker, consente al coordinatore di rilevare i nodi attivi nel sistema.
- `bonci/videoPlayerN/triggered`: pubblicato da una stazione video al rilevamento di presenza, segnala l'intenzione della guida di avviare la riproduzione.
- `bonci/videoPlayerN/ended`: inviato da una stazione video al termine della riproduzione, per notificare al coordinatore la conclusione del contenuto.
- `bonci/shutdown`: comando di emergenza per lo spegnimento completo del sistema; richiede successivamente il riavvio manuale dei dispositivi.
- `bonci/go-sleep`: attiva la modalità standby, in cui i dispositivi disattivano le funzioni principali mantenendo solo l'ascolto MQTT.
- `bonci/wake-up`: riporta il sistema dalla modalità standby alla piena operatività.
- `bonci/plafonieraN/luminosità`: usato per controllare le plafoniere.
- `bonci/audioPlayer/volume`: regola il comportamento della stazione audio.
- `shellyID/rpc`: topic predefinito nei dispositivi Shelly per il controllo remoto; utilizza un payload JSON con campo `ON` settato a `true` o `false`.

Nota:

- Il carattere `*` rappresenta un qualunque nome di dispositivo, ad esempio `videoPlayer1`, `audioPlayer`, `plafoniera3`, ecc.
- Il simbolo `N` indica un numero intero associato a una specifica istanza (es. `videoPlayer1`, `videoPlayer2`, ..., `videoPlayer4`).

Questa strategia permette al sistema di adattarsi dinamicamente al flusso della visita, gestendo il coordinamento tra dispositivi in modo efficace, reattivo e resiliente.

3.3.3 Sequenza di comunicazione tra dispositivi

Per fornire una rappresentazione del comportamento del sistema durante una visita, è stato realizzato un diagramma di sequenza UML. Questo schema descrive in modo astratto lo scambio di messaggi tra i componenti coinvolti, senza entrare nel dettaglio del protocollo di comunicazione MQTT o della struttura dei messaggi.

Il diagramma adotta infatti un approccio funzionale e astratto, con l'obiettivo di rappresentare la logica di comunicazione tra i dispositivi senza entrare nei dettagli tecnici dell'implementazione.

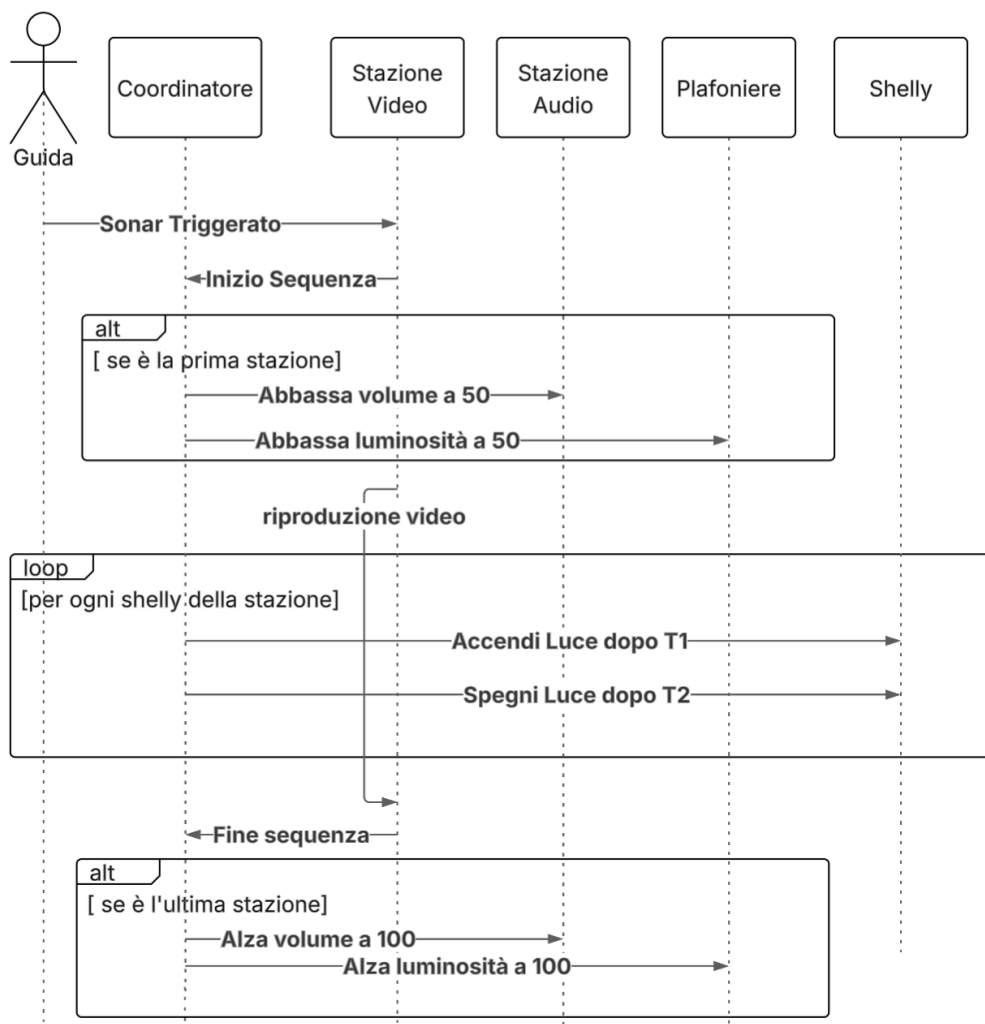


Figura 3.5: Diagramma di sequenza concettuale delle interazioni tra dispositivi durante la visita guidata

Capitolo 4

Implementazione del sistema

In questo capitolo viene descritta l'implementazione pratica del sistema, illustrando nel dettaglio le soluzioni adottate per realizzare i sottosistemi descritti nella fase di progettazione. Vengono presentate le componenti hardware e software utilizzate, insieme all'organizzazione dei processi, alle logiche di comunicazione tra i dispositivi e alle tecniche impiegate per garantire integrazione nell'ambiente museale.

4.1 Implementazione del coordinatore

Il coordinatore centrale è implementato su un Raspberry Pi 3 model B e svolge tre funzioni fondamentali:

- Creazione dell'hotspot Wi-Fi, che costituisce una rete locale isolata;
- Avvio del broker MQTT, che gestisce la comunicazione tra tutti i nodi del sistema;
- Esecuzione di uno script di coordinamento, che regola il comportamento dei dispositivi in base agli eventi della visita.

4.1.1 Struttura della rete e servizio Mqtt

Per la creazione della LAN, viene creato un **Access Point Wi-Fi** tramite il demone **Network Manager**, utilizzando l'interfaccia wireless wlan0, configurato sulla banda a 2.4GHz e fissato sul canale 6 per garantire stabilità e compatibilità con tutti i dispositivi client (ESP32, Raspberry Pi, PC, ecc.). L'SSID è personalizzato e protetto con chiave WPA2-PSK.

Parallelamente, viene avviato un **broker MQTT** basato su **Mosquitto**, che ascolta sulla porta standard 1883 e accetta connessioni da tutti i nodi

della rete senza autenticazione (modalità `allow_anonymous`), trattandosi di un sistema all'interno di una rete privata.

I dettagli pratici relativi alla configurazione dell'Access Point e alla configurazione personalizzata del broker Mosquitto saranno riportati nel capitolo successivo, nella sezione *Configurazione del coordinatore centrale*, insieme ai comandi `nmcli` e ai file di configurazione utilizzati.

4.1.2 Applicazione di orchestrazione logica

L'applicazione di orchestrazione è sviluppata in **Java** ed eseguita localmente sul Raspberry Pi che funge da coordinatore centrale. Essa si connette al broker Mosquitto in esecuzione sulla stessa macchina (`localhost:1883`) e si occupa della logica di sincronizzazione tra i sottosistemi tramite messaggi MQTT.

Il progetto è strutturato come un'applicazione Maven con le seguenti dipendenze principali:

- `Vert.x MQTT client`, per la gestione della comunicazione MQTT;
- `Vert.x core`, per l'esecuzione asincrona e la gestione dei timer.

Struttura generale

Il punto di ingresso è la classe `Main`, che inizializza un'istanza di `Vertx` e crea l'oggetto `MqttService`:

```
Vertx vertx = Vertx.vertx();  
new MqttService(vertx, "localhost", 1883);
```

La classe `MqttService` si occupa di:

- connettersi al broker MQTT;
- sottoscrivere i topic rilevanti;
- gestire la ricezione e l'interpretazione dei messaggi;
- pubblicare comandi di controllo verso gli altri dispositivi.

Sottoscrizione ai topic

Al momento della connessione, l'applicazione si sottoscrive a una serie di topic per monitorare e controllare l'ambiente:

```
client.subscribe("smartroom/+/data", 0);
client.subscribe("smartroom/+/videoPlayer/triggered", 0);
client.subscribe("smartroom/+/videoPlayer/ended", 0);
client.subscribe("smartroom/shutdown", 0);
client.subscribe("smartroom/go_sleep", 0);
client.subscribe("smartroom/wake_up", 0);
```

Gestione dei messaggi

Alla ricezione di un messaggio MQTT, l'applicazione estrae il payload, il topic e l'identificativo del dispositivo mittente:

```
String payload = message.payload().toString("UTF-8");
JsonObject data = new JsonObject(payload);
String topic = message.topicName();
String deviceId = data.getString("deviceId");
```

Dopo l'estrazione, il sistema filtra i messaggi in base al topic e attiva un handler specifico, pubblicando comandi su altri topic MQTT secondo la logica prevista.

Ad esempio, nel caso in cui venga ricevuto un messaggio su un topic contenente la stringa "ended", l'applicazione verifica che il dispositivo mittente sia l'ultima stazione video. In tal caso, pubblica i comandi necessari sui topic delle plafoniere e dell'audio per ripristinare l'illuminazione e il volume ambientale:

```
if (topic.contains("ended")) {
    if(deviceId.equals("videoPlayer4")){
        //Ultima stazione video terminata
        publish(plafTopic, "LIGHT_UP");
        publish(audioTopic, "100");
    }
}
```

Comandi Shelly temporizzati

Quando un video viene avviato e il coordinatore centrale riceve un messaggio MQTT su un topic contenente la stringa "triggered", il payload può includere una lista di luci Shelly da controllare. Per ciascuna luce è possibile specificare:

- l'identificativo della Shelly (id);

- un ritardo prima dell'accensione (`onAfter`);
- una durata di accensione prima dello spegnimento automatico (`offAfter`).

Il topic su cui viene pubblicato il comando è costruito concatenando l'id della Shelly e il suffisso `/rpc`, come richiesto dalle API ufficiali di Shelly.

Per ogni luce inclusa nel messaggio, il sistema imposta due timer indipendenti, sfruttando le funzionalità asincrone offerte da Vert.x:

```
String lightTopic = lightId + "/rpc";

vertx.setTimer(delaySeconds * 1000L, tid -> {
    publish(lightTopic, accendi.encode());
});
vertx.setTimer(offAfter * 1000L, offTid -> {
    publish(lightTopic, spegni.encode());
});
```

Il primo timer invia il comando di accensione dopo `onAfter` secondi, mentre il secondo invia il comando di spegnimento dopo `offAfter` secondi. In questo modo ogni luce può essere controllata in modo autonomo rispetto alle altre.

Di seguito un esempio del comando per accendere una luce:

```
JsonObject accendi = new JsonObject()
    .put("id", 1)
    .put("src", "server")
    .put("method", "Switch.Set")
    .put("params", new JsonObject()
        .put("id", 0)
        .put("on", true));
```

Il comando per spegnere la luce è del tutto analogo, ma il parametro `on` viene impostato su `false`.

4.2 Implementazione delle stazioni video

Le stazioni video rappresentano il fulcro dell'esperienza interattiva nella stanza Bonci. Ogni stazione è responsabile della riproduzione di contenuti multimediali e dell'interazione con l'ambiente circostante, attraverso l'utilizzo di sensori di prossimità e sistemi di controllo remoto delle luci decorative.

4.2.1 Hardware

Ogni stazione video è composta da una serie di dispositivi hardware selezionati con criteri di efficienza, affidabilità e minimo impatto visivo sull'ambiente museale. Tutti i componenti sono stati integrati in modo discreto nell'allestimento, rispettando i vincoli estetici e funzionali dell'esposizione.

Componenti della stazione

L'architettura hardware della stazione comprende:

- **Raspberry Pi 3 Model B+**: costituisce l'unità centrale di elaborazione. La scelta del modello 3B+ è motivata dal buon compromesso tra prestazioni, disponibilità di GPIO, stabilità della connessione Wi-Fi integrata e compatibilità HDMI-CEC per controllo automatico dello schermo. Il Raspberry è alloggiato all'interno di un case plastico dotato di ventola integrata e dissipatori in alluminio, per garantire un raffreddamento adeguato.
- **Display**: consente la riproduzione dei contenuti multimediali e presenta la modalità HDMI-CEC. Le dimensioni sono state scelte in base alla distanza di visione prevista per il pubblico.
- **Sensore a ultrasuoni HC-SR04**: rileva la presenza della guida o dei visitatori in prossimità della stazione. È utilizzato per attivare la riproduzione video senza necessità di tocco.
- **Modulo Shelly Plug IT**: consente il controllo remoto dell'illuminazione locale associata alla stazione. L'integrazione via Wi-Fi e la compatibilità con prese italiane ne facilitano l'installazione non invasiva.
- **Luci locali decorative**: collegate alle Shelly, sono utilizzate per enfatizzare visivamente l'attivazione della postazione.

Tutti i dispositivi della stazione sono posizionati all'interno di un cassetto in legno appeso a parete. La cornice frontale lascia visibile solamente lo schermo, mentre tutti gli altri elementi risultano completamente nascosti. Il cassetto è apribile e consente un facile accesso per interventi tecnici o aggiornamenti futuri. Al suo interno i cablaggi e i moduli sono fissati in modo stabile, e il sensore di prossimità è reso visibile grazie a due aperture laterali che ne permettono il corretto funzionamento. L'alimentazione del sistema avviene tramite un'apertura inferiore, attraverso la quale passano i cavi di alimentazione del display e del Raspberry Pi; la stessa apertura favorisce anche il ricircolo dell'aria, contribuendo alla dissipazione del calore prodotto all'interno della struttura.

Circuito elettrico

Il sensore HC-SR04 è montato su una breadboard nascosta all'interno dell'alloggiamento, ed è collegato:

- **TRIG** collegato al GPIO 23 (pin fisico 16).
- **ECHO** collegato al GPIO 24 (pin fisico 18) tramite partitore resistivo.
- **Alimentazione:** VCC a 5 V (pin 2), GND (pin 6).

Poiché il pin ECHO restituisce un segnale a 5 V, è necessario ridurlo a 3.3 V tramite un **partitore resistivo**, composto da:

- $R_1 = 1\text{ k}\Omega$
- $R_2 = 2\text{ k}\Omega$

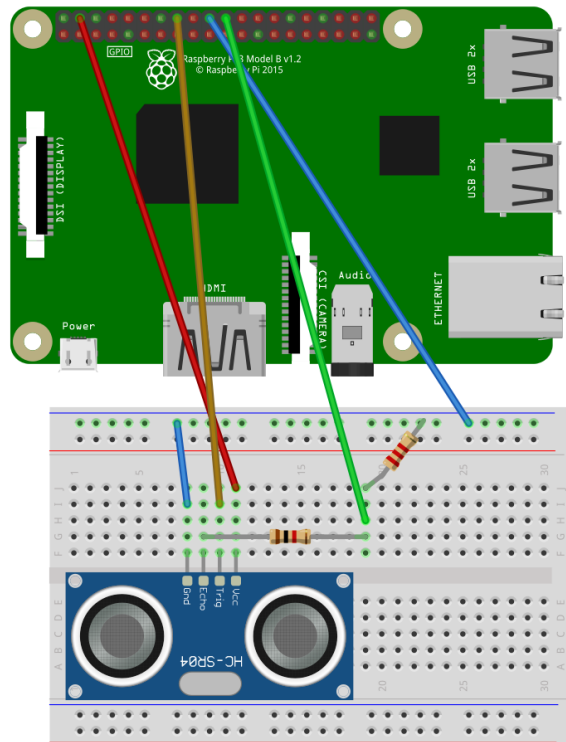


Figura 4.1: Schema elettrico della stazione video con collegamento HC-SR04 e partitore di tensione.

4.2.2 Software

Ogni stazione video del sistema è progettata per funzionare in modo autonomo su un dispositivo Raspberry Pi. Per ottenere una maggiore modularità, robustezza e capacità di gestione energetica, si è adottata un'architettura a due processi separati, ciascuno con compiti specifici ma cooperanti sullo stesso dispositivo.

Questa scelta architetturale nasce dall'esigenza di poter controllare in modo dinamico e sicuro lo stato di attività del sistema, specialmente durante le fasi di inattività, in cui è fondamentale ridurre i consumi energetici.

In particolare, durante le ore di chiusura del museo o nei momenti di inattività prolungata, è necessario mettere la stazione in modalità di risparmio energetico, che comporta:

- lo spegnimento completo dell'ambiente grafico JavaFX;
- la disattivazione delle periferiche GPIO;
- la possibilità di mantenere attiva solo la parte MQTT per continuare ad ascoltare comandi provenienti dal coordinatore centrale.

Tentativi iniziali di implementare tutto all'interno di una singola applicazione JavaFX hanno mostrato limitazioni importanti: risulta complesso e poco affidabile disattivare selettivamente le componenti grafiche e embedded lasciando attiva solo la comunicazione. Per questo motivo si è deciso di separare il software in due processi distinti

Architettura a due processi

I due processi software che compongono ogni stazione video sono:

- **mqtt-process**: è un'applicazione Java standalone, sempre attiva, che:
 - gestisce la connessione con il broker MQTT;
 - ascolta comandi remoti dal coordinatore (es. accensione, sleep, shutdown);
 - avvia o arresta il secondo processo usando un `ProcessManager`;
 - espone un server HTTP locale per ricevere notifiche da `playvideo-app`.
- **videoPlayer-process**: è un'applicazione JavaFX avviata solo su richiesta, che:
 - visualizza contenuti multimediali a schermo intero;

- interagisce con i GPIO per il rilevamento della presenza tramite sensore HC-SR04;
- comunica lo stato (inizio/fine video) al processo principale tramite HTTP.

Il processo `mqtt-process`, essendo sempre attivo, mantiene il controllo del dispositivo anche durante lo stato di inattività, e può quindi decidere di riattivare `videoPlayer-process` in risposta a un comando MQTT. La comunicazione è unidirezionale: il servizio video notifica eventi tramite chiamate HTTP al processo primario, che si occupa di inoltrarli al broker MQTT.

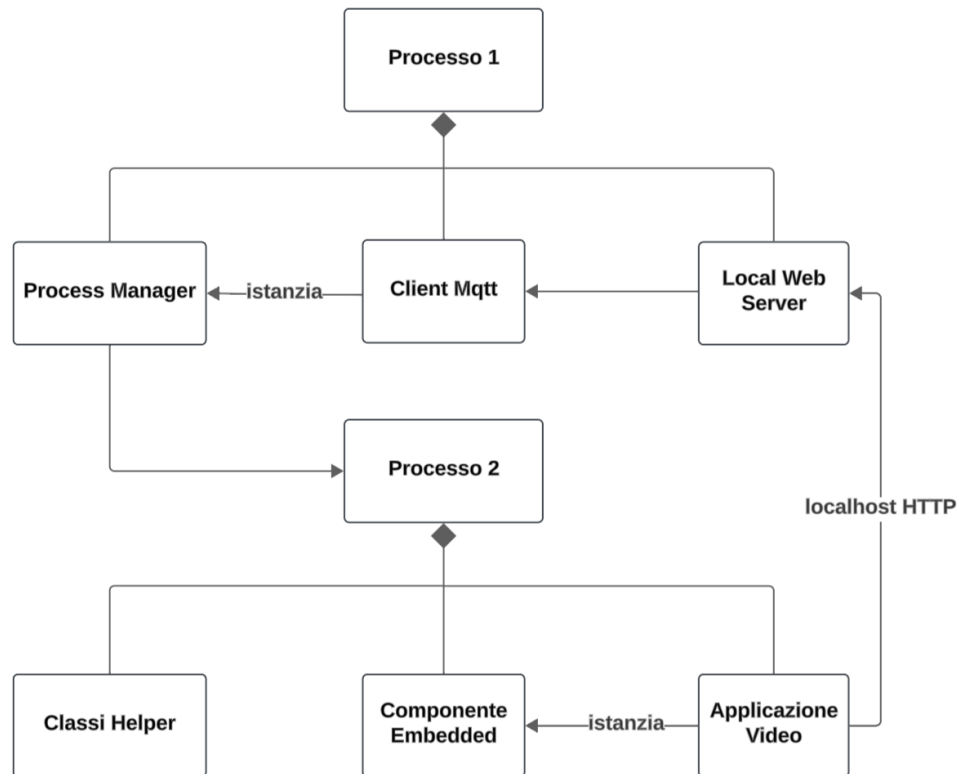


Figura 4.2: Architettura software a due processi

Processo primario: `mqtt-process`

Il processo primario, denominato `mqtt-process`, è un'applicazione Java standalone sempre attiva, progettata per fungere da interfaccia tra la componente video e il coordinatore centrale tramite protocollo MQTT. Oltre a gestire

la comunicazione con il broker, ha la responsabilità di monitorare lo stato del sistema, avviare o arrestare il processo secondario e ricevere notifiche da esso tramite HTTP.

L'intero progetto è gestito tramite **Maven** e le principali dipendenze utilizzate sono parte dell'ecosistema **Vert.x**. In particolare, il file **pom.xml** include:

- **vertx-core**: componente base per l'esecuzione asincrona e l'architettura event-loop;
- **vertx-mqtt**: modulo che fornisce il supporto nativo per client e server MQTT;
- **vertx-web**: utilizzato per esporre un'interfaccia HTTP REST, necessaria per ricevere eventi dalla componente video.

Classe Main La classe **Main** rappresenta il punto di ingresso dell'applicazione. All'avvio:

- inizializza un'istanza di **Vert.x**, il framework asincrono usato per gestire eventi e timer;
- crea un oggetto **MqttHandler**, che gestisce tutta la logica MQTT e il controllo del processo secondario;
- crea un oggetto **WebServer**, responsabile della ricezione di eventi HTTP dal processo secondario.

```
Vertx vertx = Vertx.vertx();  
MqttHandler mqttService = new MqttHandler(vertx);  
WebServer httpService = new WebServer(vertx, mqttService);
```

Classe MqttHandler La classe **MqttHandler** è responsabile della gestione della comunicazione MQTT, inclusa la connessione al broker, la sottoscrizione ai topic, la pubblicazione di messaggi e la gestione degli eventi ricevuti.

L'implementazione ruota attorno al metodo principale **attemptConnection()**, che coordina l'intera procedura. Una volta stabilita la connessione, i metodi invocati sono i seguenti:

- **subscribeToTopics()**: effettua la sottoscrizione ai topic MQTT rilevanti per ricevere i comandi dal coordinatore e implementa le funzioni di callback;

- `publishData()`: pubblica informazioni di stato all'avvio sul broker MQTT per far capire che la stazione video è attiva;
- `manager.startPlayVideoApp()`: avvia il processo secondario dell'applicazione video (GUI), assicurandosi che venga eseguito solo in presenza di connessione attiva al broker.

Segue il codice del metodo `attemptConnection()`, che incapsula tutta la logica di gestione della connessione:

```
// Connessione iniziale
public void attemptConnection() {
    client.connect(1883, "10.42.0.1", ar -> {
        if (ar.succeeded()) {
            subscribeToTopics();
            publishData();
            manager.startPlayVideoApp(true);

            client.closeHandler(v -> {
                //Connessione persa con il broker
                manager.stopPlayVideoApp();
                vertx.setTimer(10_000, id -> attemptConnection());
            });

        } else {
            attemptConnection();
        }
    });
}
```

La logica è progettata per garantire robustezza operativa: al momento della connessione, viene avviato il sottosistema video. In caso di disconnessione dal broker, la GUI viene arrestata e, dopo un intervallo di 10 secondi, viene avviato automaticamente un nuovo tentativo di connessione. Questo approccio assicura la continuità del servizio anche in presenza di interruzioni temporanee della rete.

Classe `ProcessManager` La classe `ProcessManager` è responsabile dell'avvio e arresto dell'applicazione JavaFX e della gestione del consumo energetico del dispositivo.

- `startPlayVideoApp(boolean first)`: lancia lo script `start-videoPlayer.sh`, che avvia il processo secondario. Se il parametro `first` è `true`, significa

che il Raspberry Pi è stato appena acceso e non è passato da una modalità *sleep*: in questo caso viene avviata direttamente l'applicazione. Se invece `first` è `false`, significa che il dispositivo era stato precedentemente messo in *sleep*: prima di avviare il processo, viene eseguito lo script `wakeup.sh`, che riattiva le funzionalità disattivate (riavvio dell'ambiente grafico, HDMI, CPU, ecc.).

- `stopPlayVideoApp()`: viene sempre chiamato quando il sistema riceve il comando di standby. Termina il processo JavaFX, se in esecuzione, e successivamente esegue lo script `sleep.sh`, che spegne i monitor HDMI, disattiva l'ambiente grafico, e riduce la frequenza della CPU per abbassare il consumo energetico.

Esempio semplificato di implementazione:

```
private Process playVideoProcess;

public void startPlayVideoApp(boolean first) {
    if (playVideoProcess != null && playVideoProcess.isAlive())
        return;

    try {
        if (!first) Runtime.getRuntime().exec("bash wakeup.sh");
        playVideoProcess = new ProcessBuilder("bash",
            "start-videoPlayer.sh").start();
    } catch (IOException e) {
        System.err.println("Errore" + e.getMessage());
    }
}

public void stopPlayVideoApp() {
    if (playVideoProcess != null && playVideoProcess.isAlive()) {
        playVideoProcess.destroy();
        playVideoProcess = null;
    }

    try {
        Runtime.getRuntime().exec("bash sleep.sh");
    } catch (IOException e) {
        System.err.println("Errore" + e.getMessage());
    }
}
```

Classe `WebServer` Il web server espone un'interfaccia REST su `localhost:8080` e ha il compito di ricevere notifiche dalla componente secondaria `videoPlayer-process`, senza che quest'ultima debba occuparsi direttamente della comunicazione MQTT.

In particolare, il server gestisce due endpoint:

- **POST su `/event/triggered`:** chiamato automaticamente all'inizio della riproduzione video; oltre a segnalare l'evento di attivazione, include anche una configurazione JSON relativa alle luci locali da accendere;
- **POST su `/event/ended`:** chiamato al termine della riproduzione video per segnalare la fine dell'evento.

Un esempio di handler implementato in Vert.x per la rotta `/event/triggered` è il seguente:

```
router.post("/event/triggered").handler(ctx -> {  
    String json = ctx.getBodyAsString();  
    List<LightConfig> lights = parseLights(json);  
    mqtt.publishTriggered(lights);  
    ctx.response().end();  
});
```

Come si può osservare dal codice, il corpo della richiesta contiene una configurazione JSON delle luci locali, che viene elaborata e convertita in una lista di oggetti `LightConfig`. Il server passa quindi tali configurazioni alla classe MQTT, che si occupa di pubblicarle verso i dispositivi Shelly coinvolti.

Secondo processo: `videoPlayer-process`

Il secondo processo software svolge un ruolo centrale nella gestione dell'interfaccia utente, nella riproduzione multimediale e nell'interazione con il sensore di prossimità. Si tratta di un'applicazione sviluppata in JavaFX, che include anche codice di basso livello per l'accesso ai pin GPIO tramite la libreria Pi4J.

L'intero progetto è gestito con **Maven**. Il file `pom.xml` include un insieme di dipendenze necessarie per garantire l'integrazione tra GUI, hardware e comunicazione. In particolare:

- **Pi4J core e plugin:** per l'accesso diretto all'hardware GPIO del Raspberry Pi;
- **JavaFX (base, controls, graphics, media):** per la realizzazione dell'interfaccia grafica e la riproduzione dei contenuti multimediali;

Classe `videoPlayer` La classe `videoPlayer` rappresenta il cuore dell'applicazione grafica JavaFX. Essa gestisce l'interfaccia utente e il comportamento della stazione video durante il ciclo di interazione con il visitatore. L'architettura dell'interfaccia è organizzata secondo una gerarchia di nodi JavaFX, in cui ciascun elemento corrisponde a una componente visuale o logica dell'applicazione. Di seguito è riportato uno schema della struttura gerarchica principale:

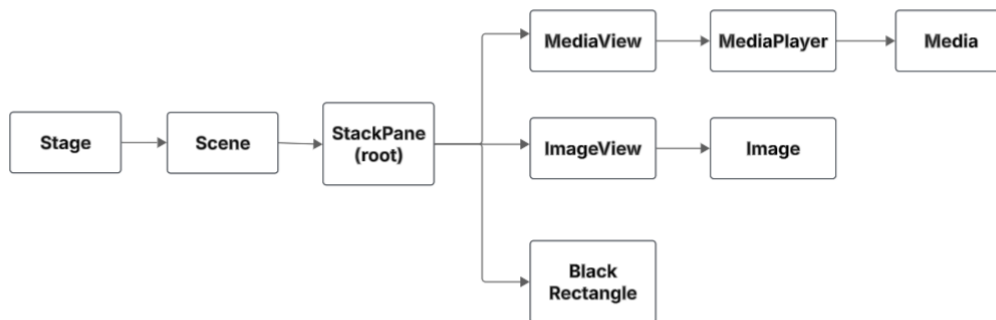


Figura 4.3: Componenti dell'applicazione JavaFx.

Nel dettaglio:

- **Stage**: rappresenta la finestra principale dell'applicazione.
- **Scene**: contiene tutti i nodi visuali visibili nella finestra.
- **StackPane**: è il nodo radice, un layout che sovrappone i nodi figli in ordine (dal più in basso al più in alto).
- **MediaView**: visualizza il video in riproduzione, gestito dal relativo **MediaPlayer**.
- **MediaPlayer**: controlla la riproduzione multimediale, ed è associato a un oggetto **Media** che incapsula il file video.
- **ImageView**: mostra una miniatura del video tramite un oggetto **Image**, utile quando il video non è in riproduzione.
- **Rectangle**: rappresenta uno sfondo nero utilizzato per transizioni o per lo stato inattivo.

La logica dell'interazione è governata da una semplice macchina a stati interna, con i seguenti stati:

- `BLACK_SCREEN`: lo schermo è inattivo (sfondo nero);
- `SHOW_THUMBNAIL`: viene mostrata la miniatura del video;
- `PLAYING_VIDEO`: è in corso la riproduzione del contenuto multimediale.

Quando il sensore rileva una presenza, viene invocato il metodo `triggered()`, che agisce in base allo stato corrente del sistema. Se il sistema si trova nello stato `SHOW_THUMBNAIL`, il video viene avviato:

```
public void triggered() {
    switch (currentState) {
        case BLACK_SCREEN:
            currentState = State.SHOW_THUMBNAIL;
            break;
        case SHOW_THUMBNAIL:
            if (mediaPlayer != null) {
                mediaPlayer.play();
                currentState = State.PLAYING_VIDEO;
            }
            break;
        case PLAYING_VIDEO:
            //Video in riproduzione
            break;
    }
    updateViewForState();
}
```

```
private void updateViewForState() {
    //Mostra o nasconde un elemento nella Scene, in base allo stato
    //del sistema
    blackScreen.setVisible(currentState == State.BLACK_SCREEN);
    thumbnailView.setVisible(currentState == State.SHOW_THUMBNAIL);
    mediaView.setVisible(currentState == State.PLAYING_VIDEO);
}
```

All'avvio della riproduzione, il sistema notifica il server HTTP, indicando che la riproduzione del video è iniziata. Al termine della riproduzione, il sistema torna automaticamente allo stato precedente, mostrando nuovamente la miniatura. Inoltre, viene notificato un evento HTTP al processo principale per indicare che la riproduzione è terminata:

```
mediaPlayer.setOnPlaying(() -> notifyEvent("triggered"));
mediaPlayer.setOnEndOfMedia(() -> {
    notifyEvent("ended");
    currentState = State.SHOW_THUMBNAIL;
    updateViewForState();
});
```

```
private void notifyEvent(String event) {
    try {
        URL url = new URL("http://localhost:8080/event/" + event);
        HttpURLConnection conn = (HttpURLConnection)
            url.openConnection();
        conn.setRequestMethod("POST");
        conn.setRequestProperty("Content-Type", "application/json");
        conn.setDoOutput(true);
        conn.getOutputStream().write(json.getBytes());

    } catch (IOException e) {
        System.err.println("Errore" + e.getMessage());
    }
}
```

Classe `EmbeddedSubsystem` Questa classe si occupa della gestione dei componenti fisici connessi al Raspberry Pi. In particolare, inizializza un contesto Pi4J e crea un rilevatore di distanza utilizzando un sensore ad ultrasuoni HC-SR04. L'inizializzazione avviene all'interno della classe `videoPlayer`, che invoca il metodo `spawnSonarDetector()` al momento dell'avvio dell'applicazione.

Il metodo riceve come parametro una `Runnable` da eseguire quando viene rilevata una presenza: in questo caso, si tratta del metodo `triggered()` già illustrato nella classe `videoPlayer`, responsabile dell'avvio della riproduzione video.

```
public void spawnSonarDetector(Runnable task) throws Exception {
    detector = new DistanceDetector(pi4j, 15, task);
    detector.start();
}
```

Classe DistanceDetector Questa classe esegue ciclicamente misurazioni della distanza, rilevando la presenza di una persona, se due misurazioni consecutive sono comprese nella soglia limite, viene invocata la callback passata dal main:

```
while (true) {
    double d = getDistance();

    if (d < maxDistance && d > minDistance) {
        validCount++;
        if (validCount >= 2) {
            task.run();
            validCount = 0;
            Thread.sleep(5000);
        } else {
            Thread.sleep(150);
        }
    } else {
        validCount = 0;
        Thread.sleep(150);
    }
}
```

Nota importante. La Runnable passata (il metodo `triggered()` della classe `videoPlayer`) può contenere operazioni che modificano l'interfaccia grafica JavaFX, come l'avvio del `MediaPlayer`.

Poiché il thread che esegue il ciclo di rilevamento non è il JavaFX Application Thread, tutte le operazioni che coinvolgono la GUI devono essere eseguite all'interno di un blocco `Platform.runLater(...)`, che garantisce l'esecuzione sicura sul thread corretto. In caso contrario, si rischiano eccezioni di tipo `IllegalStateException` o comportamenti non deterministici durante l'aggiornamento della GUI.

Classe UsbMediaScanner Questa classe viene utilizzata all'inizio della classe `videoPlayer` per la ricerca dei dispositivi USB montati su `/media/` e `/mnt/` contenenti i contenuti necessari (.mp4, .jpg, light.json)

Solo dopo che tutti gli elementi richiesti sono stati trovati, l'applicazione JavaFX può avviare la GUI utilizzando i contenuti rilevati per la riproduzione multimediale.

4.3 Implementazione della stazione audio

4.3.1 Hardware

La stazione audio è realizzata con un **Raspberry Pi 3 Model B+**, collegato direttamente alla cassa audio presente nella stanza Bonci mediante un cavo jack da 3.5 mm. Non è richiesto alcun circuito elettronico aggiuntivo: l'interfacciamento avviene esclusivamente tramite l'uscita audio analogica integrata.

4.3.2 Software

Il software della stazione audio è sviluppato in **Java**, utilizzando JavaFX per la riproduzione multimediale e la libreria Vert.x MQTT client per la connessione al broker MQTT.

Dipendenze principali

Il progetto è gestito con **Maven** e tra le dipendenze principali dichiarate nel `pom.xml` troviamo:

- **Vert.x Core** e **Vert.x MQTT Client**, per la comunicazione MQTT e la gestione asincrona degli eventi;
- **JavaFX Media** per la riproduzione multimediale, compilato per architettura linux-aarch64;

Architettura software

Il software è composto da due classi principali:

- **Main**, che gestisce:
 - l'inizializzazione dell'istanza MQTT con Vert.x;
 - la sottoscrizione ai topic MQTT;
 - la ricezione e gestione dei messaggi MQTT;
 - la logica di riconnessione in caso di disconnessione;
 - l'invocazione della logica di riproduzione tramite **AudioPlayer**.
- **AudioPlayer**, che incapsula la gestione della riproduzione audio, fornendo metodi come `start()`, `pause()`, `resume()`, `stop()`, `setVolume()` e `getStatus()` basati su **MediaPlayer** di JavaFX.

Connessione al broker MQTT e gestione eventi

La connessione al broker MQTT viene stabilita all'avvio dell'applicazione mediante il client Vert.x. In caso di successo, viene effettuata la sottoscrizione al topic `smartroom/audio/volume` e viene pubblicato un messaggio informativo su `smartroom/audio/data`, contenente alcune informazioni di sistema.

Segue una versione semplificata del codice di connessione:

```
client.connect(brokerPort, brokerHost, s -> {
    if (s.succeeded()) {
        client.subscribe("smartroom/audio/volume", 1);

        client.publish("smartroom/audio/data",
            Buffer.buffer(onlinePayload.encode()),
            MqttQoS.AT_LEAST_ONCE,
            false, false);

        client.publishHandler(message -> {
            //Vedi sezione successiva
        });

        client.closeHandler(v -> {
            //Vedi sezione successiva
        });
    } else {
        attemptConnection();
    }
});
```

Gestione dei messaggi

Alla ricezione di un messaggio MQTT, il payload viene interpretato nel metodo `handleMessage()`, eseguito all'interno di `Platform.runLater()` per garantire che ogni interazione con JavaFX avvenga sul *JavaFX Application Thread*, come richiesto dal framework:

```
client.publishHandler(message -> {
    String topic = message.topicName();
    String payload = message.payload().toString();
    Platform.runLater(() -> handleMessage(payload));
});
```

Nel metodo `handleMessage()`, l'applicazione reagisce in base al contenuto del messaggio:

```
private void handleMessage(String payload) {
    switch (payload) {
        case "TRIGGERED": triggered(); break;
        case "50": volume(0.5); break;
        case "100": volume(1.0); break;
        case "0": volume(0.0); break;
        default:
            System.out.println("Messaggio non riconosciuto: " +
                               payload);
    }
}
```

I comandi "50", "100" e "0" modificano il volume. Il comando "TRIGGERED" alterna tra avvio, pausa e ripresa del brano in base allo stato corrente del player:

Gestione disconnessioni

In caso di perdita della connessione con il broker MQTT, l'applicazione reagisce in modo automatico avviando un nuovo tentativo di connessione ogni 10 secondi tramite un timer di Vert.x. Durante questo intervallo, la riproduzione audio viene interrotta per evitare comportamenti incoerenti:

```
client.closeHandler(v -> {
    System.err.println("Connessione MQTT persa. Riprovo tra 10s...");
    vertx.setTimer(10_000, id -> attemptConnection());
    Platform.runLater(() -> player.stop());
});
```

Anche in questo caso la chiamata a `Platform.runLater()` è necessaria affinché lo stop del lettore multimediale avvenga sul `JavaFX Application Thread`.

Una volta ristabilita la connessione, l'applicazione riprende il normale funzionamento.

Classe `AudioPlayer`

La classe `AudioPlayer` incapsula tutta la logica relativa alla riproduzione audio e si basa su `MediaPlayer` di JavaFX. Il file audio viene caricato dalla cartella `audio/`, riprodotto in loop continuo, e può essere facilmente sostituito modificando il contenuto della directory, senza necessità di aggiornare il codice:

```
File file = new File("audio/" + filename);
Media media = new Media(file.toURI().toString());
MediaPlayer mediaPlayer = new MediaPlayer(media);
mediaPlayer.setCycleCount(MediaPlayer.INDEFINITE);
mediaPlayer.play();
```

La classe espone inoltre tutti i metodi necessari per manipolare la traccia audio, i quali vengono invocati dalla classe `Main` in risposta ai comandi ricevuti via MQTT. In particolare, fornisce:

- `start(String filename)` – avvia la riproduzione del file indicato;
- `stop()` – interrompe completamente l’audio;
- `setVolume(double)` – imposta il volume del player;

4.4 Implementazione delle plafoniere intelligenti

Le plafoniere intelligenti sono state progettate per essere controllabili in remoto tramite segnali a infrarossi (IR), riproducendo i comandi del telecomando originale. Ogni plafoniera contiene al suo interno un microcontrollore **ESP32-S3** con un trasmettitore IR connesso a uno dei pin GPIO. Questo modulo riceve comandi tramite protocollo MQTT e invia il segnale corrispondente alla plafoniera.

4.4.1 Hardware

Componenti principali

- **ESP32**: microcontrollore dual-core con supporto Wi-Fi integrato, scelto per le sue dimensioni compatte e la possibilità di programmazione via USB.
- **Trasmettitore IR LED** collegato al pin GPIO 4, opportunamente dimensionato per garantire la copertura angolare all’interno del diffusore della plafoniera.
- **Alimentatore AC-DC 5V**: installato all’interno della plafoniera stessa e collegato alla linea AC che già alimenta i LED.

Installazione e cablaggio

Il modulo ESP32 è stato installato direttamente all'interno della struttura della plafoniera. L'alimentazione del microcontrollore è stata prelevata utilizzando un derivatore a valle del driver LED: è stato infatti inserito un alimentatore AC-DC da 230V a 5V, in grado di fornire energia sufficiente all'ESP32.

Il trasmettitore IR è stato posizionato in modo da avere visibilità diretta verso il ricevitore IR integrato nella plafoniera. I collegamenti sono stati effettuati come segue:

- DATA → GPIO 4 dell' ESP32
- GND → GND dell'ESP32
- VCC → 5V dell'ESP32

Il circuito non richiede componenti aggiuntivi (come transistor o resistenze di pilotaggio) in quanto l'ESP32 riesce a fornire corrente sufficiente per pilotare direttamente un singolo LED IR, grazie alla modulazione a 38 kHz supportata internamente.

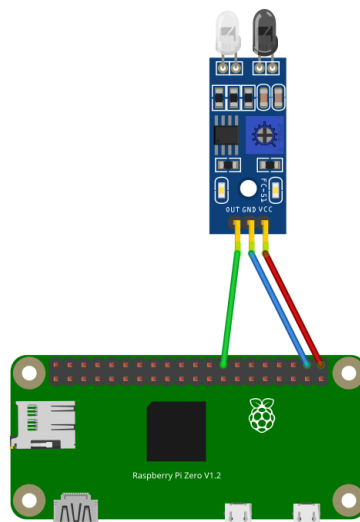


Figura 4.4: Schema elettrico del modulo di trasmissione IR

Nota: i componenti mostrati in figura sono indicativi e non corrispondono ai modelli effettivamente utilizzati nel sistema.

4.4.2 Software

Il firmware della plafoniera è stato sviluppato in **C++** utilizzando **PlatformIO** con framework Arduino. Il dispositivo ESP32 riceve comandi via MQTT e li converte in segnali infrarossi NEC per controllare l'illuminazione.

Librerie utilizzate

- IRremoteESP8266: per la trasmissione di segnali IR;
- PubSubClient: per la connessione al broker MQTT e la gestione dei topic;
- ArduinoJson: per creare e inviare messaggi JSON.

Connessione Wi-Fi e MQTT

All'avvio, il dispositivo si connette alla rete Wi-Fi locale creata dal coordinatore centrale. Una volta stabilita la connessione, il dispositivo si connette al broker MQTT e si sottoscrive al topic `smartroom/plafoniera/luminosità`:

```
// Connessione Wi-Fi
void setup_wifi()
{
    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED)
    {
        delay(500);
    }
}

// Connessione al broker MQTT
client.setServer(mqtt_server, 1883);
client.setCallback(callback);
client.connect("ESP32Client");
client.subscribe("smartroom/plafoniera/luminosità");
```

Messaggi in ingresso e Trasmissione IR

Il dispositivo ascolta il topic `smartroom/plafoniera/luminosità` su cui può ricevere i seguenti comandi stringa:

- ON – accensione della plafoniera (0x807F807F);

- OFF – spegnimento della plafoniera (0x807F807F);
- LIGHT_UP – incremento della luminosità (0x807FC03F);
- LIGHT_DOWN – decremento della luminosità (0x807F10EF).

Ogni comando viene tradotto in un segnale IR (a infrarossi), inviato tramite il pin configurato nel costruttore dell'oggetto `IRsend` a cui è collegato il trasmettitore. Il metodo `sendNEC()` della libreria `IRremoteESP8266` consente di trasmettere un codice NEC a 32 bit specificando il valore e il numero di bit. Ad esempio:

```
if (command == "LIGHT_DOWN") {  
  for (int i = 0; i < 5; i++) {  
    irsend.sendNEC(0x807F10EF, 32);  
    delay(500);  
  }  
}
```

Il **protocollo NEC** è uno dei più diffusi standard per la trasmissione di segnali a infrarossi tra telecomandi e dispositivi elettronici, ed è utilizzato anche dalle plafoniere della stanza Bonci.

Ogni comando trasmesso secondo questo protocollo è costituito da una sequenza codificata di 32 bit, suddivisi come segue:

- 8 bit per l'indirizzo del dispositivo;
- 8 bit per il complemento dell'indirizzo (controllo di integrità);
- 8 bit per il comando;
- 8 bit per il complemento del comando.

Questa sequenza viene rappresentata come un valore esadecimale.

Messaggi in uscita

Dopo la connessione al broker, il dispositivo invia un messaggio JSON sul topic `smartroom/plafoniera/data` per segnalare la propria disponibilità:

```
StaticJsonDocument<200> doc;  
doc["online"] = true;  
doc["deviceId"] = "plafoniera";  
doc["ip"] = WiFi.localIP().toString();  
doc["timestamp"] = millis();
```

```
char buffer[256];
serializeJson(doc, buffer);
client.publish("smartroom/plafoniera/data", buffer);
```

Robustezza e riconnessione

Nel ciclo principale, il firmware verifica costantemente sia la connessione MQTT sia quella Wi-Fi. In caso di disconnessione da uno dei due, viene eseguita una procedura di riconnessione automatica:

```
void loop() {
// Verifica connessione Wi-Fi
if (WiFi.status() != WL_CONNECTED) {
  setup_wifi();
}

// Verifica connessione MQTT
if (!client.connected()) {
  reconnect();
}

client.loop();
}
```

Il metodo `client.loop()` viene chiamato ad ogni ciclo per mantenere attiva la comunicazione con il broker e ricevere i messaggi MQTT. In assenza di questa chiamata, la callback non verrebbe mai eseguita. Grazie a questa architettura, il sistema è in grado di recuperare automaticamente anche da disconnessioni temporanee della rete Wi-Fi o del broker, senza richiedere un riavvio manuale.

4.5 Implementazione del Modulo di acquisizione IR

Durante la fase di configurazione iniziale del sistema è stato necessario acquisire i codici IR originali del telecomando fornito con le plafoniere, per poterli replicare in seguito via software attraverso i moduli trasmettenti ESP32. A questo scopo è stato realizzato un modulo di acquisizione temporaneo basato su ESP32 e un ricevitore IR.

Una volta acquisiti tutti i codici necessari per controllare le funzioni delle plafoniere (accensione, spegnimento, aumento/diminuzione luminosità), il modulo è stato dismesso e rimosso dall'infrastruttura attiva.

I codici salvati sono stati poi integrati nel firmware delle singole plafoniere intelligenti, consentendo a ciascun ESP32 di replicare in modo autonomo il comportamento del telecomando.

4.5.1 Hardware

Il modulo di acquisizione è stato realizzato utilizzando un microcontrollore **ESP32**, dotato di un pin digitale collegato a un ricevitore IR.

- **Ricevitore IR:** collegato al pin digitale GPIO 15, consente la lettura dei segnali emessi dal telecomando.
- **Pulsante fisico:** connesso al pin GPIO 4, utilizzato per attivare la registrazione dei codici in modo controllato.
- **Alimentazione:** il modulo è stato alimentato via USB durante l'intera fase di test, in quanto si tratta di un dispositivo temporaneo e non integrato nella rete finale.

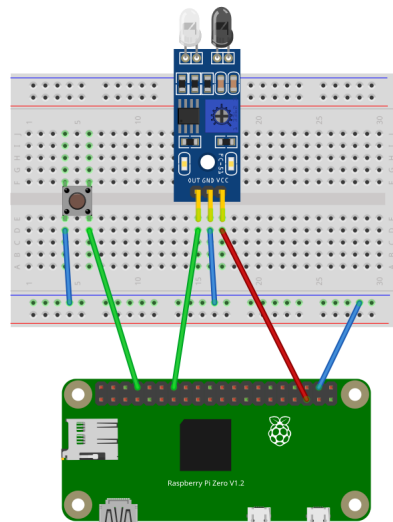


Figura 4.5: Schema elettrico del modulo di ricezione IR

Nota: i componenti mostrati in figura sono indicativi e non corrispondono ai modelli effettivamente utilizzati nel sistema.

4.5.2 Software

Il software è stato sviluppato in C++ utilizzando **PlatformIO** con **Arduino framework**, ed è basato sulla libreria `IRremoteESP8266` per la gestione della decodifica IR.

Il funzionamento è semplice: premendo il pulsante, l'ESP32 avvia la lettura per alcuni secondi. Se durante quel periodo riceve un codice valido (non UNKNOWN), lo salva in memoria e visualizza l'output sulla porta seriale. Per acquisire un codice IR, è sufficiente puntare il telecomando originale della plafoniera verso il ricevitore a infrarossi e premere il pulsante desiderato. Segue un estratto significativo del codice sorgente:

```
IRrecv irrecv(PIN_RECV);
decode_results results;

void loop() {
  if (digitalRead(BUTTON_RECV) == LOW) {
    if (irrecv.decode(&results)) {
      if (results.decode_type != UNKNOWN) {
        Serial.println("== Codice salvato ==");
        Serial.print("Protocollo: ");
        Serial.println(typeToString(results.decode_type));
        Serial.print("Codice HEX: 0x");
        Serial.println(results.value, HEX);
        Serial.print("Bit: ");
        Serial.println(results.bits);
      }
      irrecv.resume();
    }
  }
}
```

Un esempio di output visualizzato durante l'acquisizione è il seguente:

```
== Codice salvato ==
Protocollo: NEC
Codice HEX: 0x20DF10EF
Bit: 32
```

Capitolo 5

Configurazione dei dispositivi

In questo capitolo vengono descritte nel dettaglio le operazioni necessarie per configurare correttamente i dispositivi utilizzati all'interno del sistema. Tali operazioni includono l'installazione del sistema operativo, la configurazione dei servizi generali e la predisposizione dell'ambiente Java per l'esecuzione dei componenti software sviluppati.

5.1 Configurazioni generali

Questa sezione descrive le configurazioni iniziali comuni a tutti i dispositivi Raspberry Pi utilizzati nel sistema. Le operazioni presentate non riguardano i microcontrollori ESP32, i quali seguono una logica di configurazione separata. Le configurazioni qui illustrate sono fondamentali per predisporre l'ambiente software necessario all'esecuzione delle applicazioni Java sviluppate, oltre a semplificare il trasferimento dei file e la connessione alla rete.

5.1.1 Installazione del sistema operativo

Il sistema operativo utilizzato per tutti i Raspberry Pi è Raspberry Pi OS (64-bit), versione consigliata per la versione 3 Model B+.

L'installazione può essere effettuata tramite il software ufficiale Raspberry Pi Imager, disponibile sul sito <https://www.raspberrypi.com/software/>. Una volta selezionato il sistema operativo, si può procedere con il flashing della scheda SD e l'avvio del dispositivo.

5.1.2 Installazione di Samba

Per facilitare lo sviluppo e la condivisione dei file tra Raspberry Pi e un computer, è stato installato e configurato Samba, un software che consente la

condivisione di directory in rete locale.

Creazione della cartella condivisa

Si crea una cartella sul desktop del Raspberry Pi, da rendere accessibile via rete:

```
mkdir /home/villasilvia/Desktop/condivisa
```

Installazione di Samba

Per installare Samba e i relativi tool di configurazione:

```
sudo apt install samba samba-common-bin
```

Configurazione della condivisione

Modificare il file di configurazione di Samba:

```
sudo nano /etc/samba/smb.conf
```

Aggiungere in fondo al file la seguente configurazione:

```
[condivisa]
path = /home/villasilvia/Desktop/condivisa
browseable = yes
writeable = yes
guest ok = yes
public = yes
create mask = 0777
directory mask = 0777
force user = villasilvia
```

Riavviare il servizio per rendere effettive le modifiche:

```
sudo systemctl restart smbd
```

Accesso alla cartella da PC

Dal PC, è possibile accedere alla cartella condivisa semplicemente aprendo l'esploratore file e digitando nella barra degli indirizzi:

```
//<ip_del_raspberry>/condivisa
```

Questa configurazione ha semplificato notevolmente lo sviluppo, permettendo di copiare rapidamente il codice sorgente dal computer al Raspberry senza necessità di chiavette USB o trasferimenti tramite SSH.

5.1.3 Installazione di Java e Maven

Tutti i dispositivi necessitano dell'ambiente Java per l'esecuzione del software e di Maven per la compilazione del progetto.

```
sudo apt install openjdk-17-jdk
```

```
sudo apt install maven
```

5.1.4 Connessione alla rete

Questo passaggio va effettuato esclusivamente sui Raspberry Pi delle stazioni audio e video, in quanto è il dispositivo coordinatore a mettere a disposizione l'hotspot Wi-Fi locale a cui gli altri si devono connettere. La connessione viene stabilita manualmente per garantire maggiore affidabilità anche in assenza di interfacce grafiche.

Disattivare Network Manager e Dhcpd

Per garantire il corretto funzionamento della connessione all'hotspot del coordinatore e una gestione stabile dell'interfaccia di rete, è necessario disabilitare completamente il servizio NetworkManager. Eseguire i seguenti comandi:

```
sudo systemctl stop NetworkManager  
sudo systemctl disable NetworkManager
```

In aggiunta, disattivare anche il demone `dhcpcd`:

```
sudo systemctl stop dhcpcd  
sudo systemctl disable dhcpcd
```

In questo modo si evita qualsiasi conflitto nella gestione della rete da parte di servizi concorrenti.

Configurare il file `wpa_supplicant.conf`

Modificare o creare il file di configurazione del Wi-Fi:

```
sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
```

Inserire il seguente contenuto:

```
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
country=IT

network={
    ssid="RaspPi"
    psk="password"
    key_mgmt=WPA-PSK
}
```

Creazione del servizio di connessione manuale

Creare un nuovo servizio systemd per attivare automaticamente la connessione all'avvio:

```
sudo nano /etc/systemd/system/manual-wifi.service
```

Inserire il seguente contenuto:

```
[Unit]
Description=Connessione Wi-Fi manuale con wpa_supplicant e dhclient
After=network-pre.target
Wants=network.target

[Service]
Type=oneshot
ExecStart=/sbin/wpa_supplicant -B -i wlan0 -c
    /etc/wpa_supplicant/wpa_supplicant.conf
ExecStartPost=/sbin/dhclient wlan0
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

- **ExecStart:** avvia `wpa_supplicant` in background (`-B`) sull'interfaccia Wi-Fi `wlan0` utilizzando il file di configurazione specificato.

- **ExecStartPost**: una volta stabilita la connessione, viene eseguito `dhclient wlan0` per ottenere automaticamente un indirizzo IP dal coordinatore.

Infine, rendere attivo il servizio:

```
sudo systemctl daemon-reload
sudo systemctl enable manual-wifi.service
sudo systemctl start manual-wifi.service
```

Questa procedura garantisce una connessione stabile all'hotspot messo a disposizione dal coordinatore, anche in assenza di strumenti grafici o gestori di rete automatici.

5.2 Configurazione del dispositivo coordinatore

Il dispositivo designato come coordinatore ha bisogno di configurazioni specifiche, per permettere la gestione della rete Wi-Fi, l'esecuzione del servizio Mqtt e l'esecuzione automatica del software di logica sviluppato in Java.

5.2.1 Configurazione dell'hotspot Wi-Fi

La configurazione dell'hotspot avviene tramite NetworkManager, che consente una gestione più flessibile delle connessioni rispetto al demone `dhcpcd`, che viene disattivato. Di seguito i passaggi da seguire:

- **Installazione di NetworkManager** Aggiorna la lista dei pacchetti e installa NetworkManager:

```
sudo apt update
sudo apt install network-manager
```

- **Disattivazione di dhcpcd** Ferma ed esclude il demone `dhcpcd` per evitare conflitti con NetworkManager:

```
sudo systemctl stop dhcpcd
sudo systemctl disable dhcpcd
```

- **Abilitazione di NetworkManager e riavvio del sistema** Abilita NetworkManager all'avvio e riavvia per applicare i cambiamenti:

```
sudo systemctl enable NetworkManager  
sudo reboot
```

- **Creazione della connessione Wi-Fi in modalità Access Point**
Dopo il riavvio, crea una connessione Wi-Fi con SSID personalizzato:

```
nmcli connection add type wifi ifname wlan0 con-name RaspPi  
autoconnect yes ssid RaspPi
```

- **Configurazione della modalità Access Point e parametri radio**
Imposta la connessione come Access Point sulla banda 2.4 GHz e canale 6:

```
nmcli connection modify RaspPi 802-11-wireless.mode ap  
nmcli connection modify RaspPi 802-11-wireless.band bg  
nmcli connection modify RaspPi 802-11-wireless.channel 6
```

- **Condivisione della connessione (NAT)** Abilita il metodo `shared` per condividere la connessione tramite NAT:

```
nmcli connection modify RaspPi ipv4.method shared
```

- **Configurazione della sicurezza WPA-PSK** Imposta la sicurezza della rete Wi-Fi con password:

```
nmcli connection modify RaspPi  
802-11-wireless-security.key-mgmt wpa-psk  
nmcli connection modify RaspPi 802-11-wireless-security.psk  
"password"
```

- **Attivazione della connessione** Rende operativo l'hotspot:

```
nmcli connection up RaspPi
```

5.2.2 Installazione e configurazione del broker MQTT

Il broker MQTT utilizzato è Mosquitto, un demone leggero ed efficiente, adatto anche a dispositivi embedded. Di seguito i passaggi per l'installazione e la configurazione di base.

- **Installazione del broker Mosquitto** Installa Mosquitto e i relativi strumenti client:

```
sudo apt install mosquitto mosquitto-clients
```

- **Abilitazione dell'avvio automatico** Attiva Mosquitto al boot del sistema:

```
sudo systemctl enable mosquitto
```

- **Configurazione di base** La configurazione predefinita è sufficiente per test e utilizzo in reti locali. Il file di configurazione è:

```
/etc/mosquitto/mosquitto.conf
```

Un esempio di configurazione semplificata è la seguente:

```
listener 1883
allow_anonymous true
```

- `listener 1883`: apre la porta standard MQTT per accettare connessioni in ingresso;
- `allow_anonymous true`: abilita connessioni senza autenticazione (utile in ambienti di test o reti isolate);

In contesti più strutturati, si consiglia di estendere la configurazione abilitando autenticazione tramite username e password e crittografia TLS per connessioni sicure;

5.2.3 Creazione del servizio di avvio automatico della logica di coordinamento

Per eseguire automaticamente la logica di coordinamento all'avvio del Raspberry, bisogna inserire il progetto `mqttServer` all'interno della cartella creata e messa in condivisione con samba, successivamente bisogna creare il file:

```
sudo nano /etc/systemd/system/mqttServer.service
```

Con il seguente contenuto:

```
[Unit]
Description=MqttServer
After=network.target

[Service]
User=villasilvia
ExecStart=/usr/bin/java -jar
    /path_to_jar/smartroom-dashboard-1.0.0.jar
WorkingDirectory=/home/villasilvia/Desktop/condivisa/mqttServer
SuccessExitStatus=143
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

Infine, abilitare ed avviare il servizio:

```
sudo systemctl daemon-reload
sudo systemctl enable mqttServer
sudo systemctl start mqttServer
```

Questo consente al coordinatore di essere operativo automaticamente all'accensione del dispositivo.

5.3 Configurazione del dispositivo video

Il dispositivo `videoPlayer` ha il compito di proiettare contenuti video su uno schermo collegato via HDMI. Trattandosi di una postazione permanente destinata all'esperienza museale, è necessario che il Raspberry Pi:

- non entri mai in standby,
- controlli lo stato dello schermo tramite interfaccia HDMI-CEC,
- disattivi ogni tipo di notifica, inclusa quella relativa ai dispositivi USB,
- disattivi il cursore per renderlo invisibile sugli schermi,
- esegua automaticamente l'applicazione video all'avvio.

5.3.1 Impedire lo standby dell'ambiente grafico

Per evitare che lo schermo si oscuri o si spenga, è necessario modificare il file di autostart dell'ambiente grafico LXDE:

```
nano ~/.config/lxsession/LXDE-pi/autostart
```

Aggiungere le seguenti righe:

```
lxpanel --profile LXDE-pi  
pcmanfm --desktop --profile LXDE-pi  
xscreensaver -no-splash  
sh -c "sleep 5 && xset s off"  
sh -c "sleep 5 && xset -dpms"  
sh -c "sleep 5 && xset s noblank"
```

Tali comandi disattivano lo screensaver, il power management del display (DPMS) e il blanking dello schermo, assicurando che il monitor resti sempre attivo.

5.3.2 Controllo dello schermo tramite HDMI-CEC

Per controllare via software lo stato del televisore collegato via HDMI, si installa il pacchetto `cec-utils`, che fornisce comandi per accendere o spegnere lo schermo tramite il protocollo HDMI-CEC:

```
sudo apt install cec-utils
```

Questo permette al sistema di spegnere o accendere automaticamente lo schermo quando, ad esempio, le guide turistiche mandano il sistema in standby durante la chiusura serale del museo, evitando la necessità di spegnere manualmente ogni TV.

5.3.3 Disabilitazione delle notifiche USB

Nel contesto museale, è fondamentale che l'interfaccia grafica del sistema non mostri alcuna notifica quando viene collegato un dispositivo USB, al fine di garantire un'esperienza visiva pulita e priva di interruzioni per i visitatori. Su Raspberry Pi OS con ambiente Desktop (LXDE), questo comportamento predefinito è gestito dal file manager `pcmanfm`, che rileva automaticamente l'inserimento di dispositivi rimovibili e mostra popup di notifica per il montaggio e l'esplorazione del contenuto.

E' possibile disattivare queste notifiche tramite configurazione di `pcmanfm`. A tal fine, è possibile modificare il file di configurazione utente:

- Aprire il file `~/.config/pcmanfm/LXDE-pi/pcmanfm.conf`.
- Cercare le seguenti direttive:

```
mount_on_startup=1
mount_removable=1
autorun=1
```

- Modificarle come segue per disabilitare completamente il montaggio e l'autorun automatico:

```
mount_on_startup=0
mount_removable=0
autorun=0
```

Dopo aver salvato le modifiche, è consigliabile riavviare l'ambiente grafico con il comando `lxpanelctl restart` o effettuare un logout/login per applicare le modifiche.

5.3.4 Nascondere il cursore del mouse nell'interfaccia grafica

All'interno di un'installazione museale, la presenza del cursore del mouse sullo schermo rappresenta un elemento visivo indesiderato che può compromettere l'estetica e l'immersività dell'esperienza utente. Per questo motivo, si rende necessario nascondere completamente il puntatore del mouse all'interno dell'interfaccia grafica del sistema basato su Raspberry Pi OS Desktop.

Una soluzione efficace e leggera consiste nell'utilizzo dell'applicazione `unclutter`, che permette di nascondere automaticamente il cursore del mouse dopo un breve periodo di inattività, o fin dall'avvio del sistema.

Installazione di `unclutter` Il pacchetto `unclutter` è disponibile nei repository ufficiali di Raspberry Pi OS e può essere installato con il seguente comando:

```
sudo apt install unclutter
```

Avvio automatico all'avvio della sessione Per fare in modo che il cursore venga nascosto automaticamente ad ogni avvio, è necessario avviare `unclutter` come parte della sessione LXDE. Questo può essere fatto modificando il file di autostart dell'ambiente grafico:

- Modificare il file `~/.config/lxsession/LXDE-pi/autostart`.
- Aggiungere la seguente riga alla fine del file:

```
@unclutter -idle 0
```

L'opzione `-idle 0` nasconde il cursore immediatamente, senza necessità di attendere un intervallo di inattività.

5.3.5 Avvio automatico del software videoPlayer

Per avviare automaticamente il programma all'accensione del dispositivo, bisogna inserire il software del videoPlayer nella cartella condivisa con samba, viene poi creato un servizio `systemd` nell'ambiente `user` (anziché `system`), in modo da associarlo direttamente all'utente `villasilvia`. Questa modalità evita la necessità di privilegi di root e permette una gestione più flessibile del servizio.

Creazione del servizio

Creare la directory dei servizi utente (se non esiste):

```
mkdir -p ~/.config/systemd/user
```

Creare il file del servizio:

```
nano ~/.config/systemd/user/videoPlayer.service
```

Inserire il seguente contenuto:

```
[Unit]
Description=Servizio MQTT Video Client

[Service]
ExecStart=/usr/bin/java -jar
    /path_to_jar/mqtt-service-1.0-SNAPSHOT.jar
WorkingDirectory=/path_to_directory/MqttVideoClient
Restart=always
Environment=DISPLAY=:0
Environment=XAUTHORITY=/home/villasilvia/.Xauthority

[Install]
WantedBy=default.target
```

Abilitazione del servizio all'avvio

Abilitare la persistenza dell'utente anche senza login attivo, così che i servizi utente vengano eseguiti al boot:

```
loginctl enable-linger villasilvia
```

Ricaricare il gestore dei servizi utente e abilitare il servizio:

```
systemctl --user daemon-reload
systemctl --user enable videoPlayer.service
systemctl --user start videoPlayer.service
```

5.4 Configurazione del dispositivo audio

5.4.1 Avvio automatico del software audioPlayer

Anche in questo caso, per avviare automaticamente il programma all'accensione del dispositivo, bisogna inserire il software dell'audioPlayer nella cartella condivisa con samba.

Per avviarlo automaticamente all'accensione del dispositivo, è stato creato un servizio **systemd** di tipo **system**, accessibile a livello globale.

Creazione del servizio

Creare il file del servizio:

```
sudo nano /etc/systemd/system/audioPlayer.service
```

Inserire il seguente contenuto:

```
[Unit]
Description=Servizio MQTT Audio Client
After=network-online.target graphical.target
Wants=network-online.target graphical.target

[Service]
ExecStart=/bin/bash /path_to_sh/start.sh
WorkingDirectory=/home/villasilvia/Desktop/condivisa/audioPlayer
User=villasilvia
Restart=always
Environment=DISPLAY=:0
Environment=XAUTHORITY=/home/villasilvia/.Xauthority
```

```
Environment=PULSE_SERVER=unix:/run/user/1000/pulse/native

[Install]
WantedBy=multi-user.target
```

Abilitazione dell'utente al gruppo audio

Per permettere all'utente `villasilvia` di accedere correttamente al sistema audio, è necessario aggiungerlo al gruppo `audio`:

```
sudo usermod -aG audio villasilvia
```

Abilitazione e avvio del servizio

Per rendere attivo il servizio al boot, eseguire:

```
sudo systemctl daemon-reload
sudo systemctl enable audioPlayer.service
sudo systemctl start audioPlayer.service
```

5.5 Configurazione dei moduli di trasmissione e ricezione IR

Per i moduli a infrarossi utilizzati per la trasmissione e la ricezione dei segnali IR non è necessaria alcuna configurazione particolare lato software. Questi dispositivi, infatti, sono basati su microcontrollori ESP32 e funzionano in modo completamente autonomo.

È sufficiente effettuare upload del firmware sviluppato con PlatformIO, specifico per ciascun modulo, direttamente sulla memoria flash dell'ESP32 tramite cavo USB-C.

Una volta programmati, i moduli IR risultano operativi immediatamente all'accensione, senza necessità di ulteriori configurazioni o interventi manuali.

Capitolo 6

Testing del Sistema

6.1 Obiettivi e metodologia di testing

La fase di testing ha rappresentato un momento cruciale nel processo di sviluppo del sistema, permettendo di verificare che ciascun componente rispettasse i requisiti funzionali e operasse correttamente sia in modo isolato che all'interno dell'architettura distribuita. Gli obiettivi principali di questa fase sono stati:

- Validare il corretto funzionamento dei singoli moduli hardware e software;
- Verificare il corretto collegamento dei sottosistemi tramite protocollo di comunicazione MQTT;
- Simulare il comportamento del sistema in scenari realistici, con particolare attenzione alla sequenza delle operazioni e alla sincronizzazione tra dispositivi;
- Rilevare e correggere eventuali anomalie nel comportamento del sistema.

Per raggiungere questi obiettivi, è stata adottata una metodologia incrementale articolata in tre fasi distinte: test funzionali locali, test dei sottosistemi in rete con MQTT e collaudo completo in ambiente reale.

6.2 Fase 1 – Test funzionali locali

La prima fase di testing ha avuto come obiettivo la verifica del corretto funzionamento dei singoli sottosistemi in modalità completamente autonoma, senza alcuna connessione al broker MQTT o alla rete. Questo approccio

ha permesso di concentrarsi esclusivamente sulle funzionalità locali di ciascun componente, identificando e risolvendo eventuali problemi legati all'hardware, alla logica interna del software o all'interazione con l'utente.

Ogni sottosistema è stato testato separatamente, simulando gli input esterni in modo controllato e osservando il comportamento del sistema in risposta a tali stimoli.

Stazione Video

Il sottosistema Video Player è responsabile della gestione della GUI, della riproduzione dei contenuti multimediali e dell'interazione tramite sensore a ultrasuoni. Le funzionalità testate in questa prima fase sono elencate di seguito:

- **Sensore a ultrasuoni (HC-SR04):** è stato calibrato per rilevare la presenza della guida a una distanza appropriata, evitando falsi positivi dovuti a rumore ambientale o superfici riflesse.
- **Interfaccia grafica (JavaFX):** è stata verificata la corretta visualizzazione delle miniature all'avvio, il passaggio alla riproduzione video alla ricezione di un input da sonar e il ritorno automatico alla schermata iniziale al termine del video.
- **Gestione dei contenuti multimediali:** è stata testata la corretta lettura dei file da chiavette USB collegate durante l'esecuzione.
- **Disattivazione standby e sospensione:** è stato verificato che il sistema non entri mai in modalità di sospensione automatica, anche dopo ore di inattività in stato di inattività.
- **Controllo HDMI-CEC:** è stata testata la comunicazione con il display tramite interfaccia HDMI-CEC, inviando manualmente comandi per accensione e spegnimento dello schermo da terminale.
- **Rimozione delle notifiche di sistema:** è stato confermato che l'inserimento di dispositivi USB non genera alcuna finestra di notifica o popup, mantenendo un'interfaccia pulita.
- **Cursore del mouse disattivato:** l'interfaccia è stata verificata in presenza di un mouse USB collegato, assicurandosi che il puntatore non fosse visibile in alcun momento.

- **Avvio automatico dell'applicazione:** è stato confermato che l'interfaccia JavaFX viene eseguita automaticamente all'avvio del sistema operativo, senza intervento manuale, caricandosi direttamente in modalità fullscreen.

Stazione audio

La stazione audio è stata testata in modo indipendente con l'obiettivo di verificare la corretta riproduzione e manipolazione di tracce audio. In assenza di comunicazione MQTT, sono stati predisposti comandi locali (input da tastiera) per simulare la ricezione di eventi.

I test effettuati includevano:

- Verifica dell'accesso ai file audio memorizzati nella USB inserita nel Raspberry Pi;
- Avvio manuale della riproduzione, controllo del livello del volume, interruzione della riproduzione e ritorno allo stato di attesa verificati tramite input simulato;

Grazie a questi test è stato possibile confermare che l'hardware audio era correttamente configurato, e che la logica software rispondeva in maniera affidabile a input locali.

ESP32

Il modulo ESP32 dedicato al controllo delle plafoniere LED dimmerabili tramite segnali infrarossi è stato testato in modo del tutto indipendente, alimentato tramite USB e connesso a un set di pulsanti fisici, senza alcuna rete Wi-Fi attiva né MQTT.

Per ciascun pulsante collegato a un ingresso digitale dell'ESP32, era associata una funzione IR diversa:

- Pulsante 1: trasmissione del segnale di accensione (ON);
- Pulsante 2: trasmissione del segnale di spegnimento (OFF);
- Pulsante 3: aumento della luminosità (LIGHT_UP);
- Pulsante 4: diminuzione della luminosità (LIGHT_DOWN).

I test consistevano nell'attivare manualmente i pulsanti e osservare la risposta della plafoniera:

- Verifica che ogni pressione producesse l'invio del segnale IR corrispondente;
- Riconcontro visivo sullo stato della plafoniera (accensione, spegnimento, variazione di intensità);
- Verifica della distanza e dell'angolazione ottimali per la trasmissione del segnale.

Grazie a questi test si è potuto confermare il corretto cablaggio dell'hardware, l'affidabilità della trasmissione IR e la validità dei codici registrati per il modello di plafoniera in uso.

6.3 Fase 2 – Test dei sottosistemi in rete con MQTT

La seconda fase di testing ha avuto come obiettivo principale la verifica dell'integrazione dei singoli sottosistemi all'interno del sistema distribuito, basato su comunicazione tramite protocollo MQTT. Dopo aver accertato, nella fase preliminare, il corretto funzionamento locale di ciascun componente, questa fase ha previsto la valutazione del comportamento in rete, con particolare attenzione alla ricezione di messaggi MQTT e all'attivazione remota delle funzionalità già collaudate.

Ogni sottosistema è stato pertanto ri-testato individualmente, ma all'interno dell'infrastruttura completa, collegandosi alla rete Wi-Fi locale generata dall'hotspot del coordinatore centrale. Il coordinatore, ospitato su un Raspberry Pi, svolgeva contemporaneamente il ruolo di access point e broker MQTT (Mosquitto), garantendo il corretto instradamento dei messaggi tra i vari nodi.

Le verifiche hanno incluso:

- La corretta connessione del sottosistema alla rete Wi-Fi dell'hotspot del coordinatore;
- L'avvio e il corretto funzionamento del client MQTT locale e la sottoscrizione ai topic previsti;
- L'invio di messaggi MQTT da un terminale remoto (PC collegato alla stessa LAN), tramite il comando `mosquitto_pub`, al fine di simulare gli eventi generati nel sistema reale dal coordinatore centrale;
- La corretta interpretazione dei messaggi ricevuti e l'attivazione della logica locale corrispondente, precedentemente verificata in modalità locale;

- Il corretto funzionamento del servizio Mosquitto sul coordinatore, inclusa la capacità di ricevere e distribuire i messaggi ai sottosistemi connessi.

Controllo delle luci locali (Shelly)

Il controllo delle luci locali è stato realizzato tramite dispositivi Shelly, configurati per ricevere comandi attraverso il protocollo MQTT. Ogni dispositivo è identificato da un proprio topic specifico nel formato `id-shelly/rpc`, dove `id-shelly` rappresenta l'identificativo univoco assegnato al singolo modulo.

Durante la fase di test, sono stati inviati manualmente messaggi MQTT dal terminale al fine di verificare la corretta ricezione e l'elaborazione dei comandi da parte dei dispositivi. In particolare, è stato utilizzato il seguente comando per attivare la luce associata al dispositivo:

```
mosquitto_pub -h 10.42.0.1 -t shelly-disco/rpc -m '{ "id": 1, "src":  
  "server", "method": "Switch.Set", "params": { "id": 0, "on": true  
  } }'
```

Il test ha confermato che, all'invio del messaggio, il dispositivo Shelly eseguiva correttamente l'azione richiesta (accensione o spegnimento), restituendo un feedback interno e modificando lo stato della luce.

Gestione dell'audio via MQTT

La stazione audio, già testata nella fase 1 per la riproduzione locale, è stata connessa al broker MQTT e sottoposta a test. Inviando messaggi al topic `smartroom/audio/volume`, si è verificato che la stazione ricevesse correttamente i valori e attivasse la riproduzione o la modifica del volume.

```
mosquitto_pub -h 10.42.0.1 -t smartroom/audio/volume -m "TRIGGERED"  
mosquitto_pub -h 10.42.0.1 -t smartroom/audio/volume -m "50"
```

Il collegamento tra la ricezione del messaggio MQTT e la funzione di gestione audio interna è risultato stabile.

Comandi per plafoniere via ESP32

Il dispositivo ESP32 è stato configurato per connettersi via Wi-Fi e sottoscrivere al topic `smartroom/plafoniera/luminosità`. In fase 1 si era accertata la capacità del dispositivo di inviare correttamente segnali IR tramite input fisici; in questa fase è stato verificato che l'arrivo di specifici messaggi MQTT causasse l'attivazione delle stesse funzioni IR.

```
mosquitto_pub -h 10.42.0.1 -t smartroom/plafoniera/luminosita' -m  
"ON"  
mosquitto_pub -h 10.42.0.1 -t smartroom/plafoniera/luminosita' -m  
"LIGHT_UP"
```

La risposta della plafoniera è stata osservata visivamente, confermando che il collegamento tra messaggio MQTT, codice firmware e trasmissione IR era corretto.

6.4 Fase 3 – Collaudo completo in ambiente reale

La terza e ultima fase di testing ha previsto il collaudo dell'intero sistema direttamente all'interno del contesto museale, replicando fedelmente le condizioni operative reali. Questa fase ha rappresentato un momento fondamentale per la verifica complessiva del sistema, consentendo di valutare affidabilità, stabilità e coerenza funzionale attraverso l'interazione simultanea di tutti i sottosistemi coinvolti.

Collaudo nella stanza Bonci

Il collaudo iniziale è stato eseguito in configurazione ridotta, coinvolgendo una singola stazione video, una plafoniera smart, il coordinatore centrale e la stazione audio. Tale configurazione ha permesso di simulare un'esperienza utente completa su scala contenuta, utile per individuare e correggere eventuali criticità prima del dispiegamento dell'intero sistema.

Successivamente, il test è stato esteso all'infrastruttura completa, comprendente tutte e quattro le stazioni multimediali e le sei plafoniere smart installate nella stanza Bonci. Gli obiettivi principali di questa fase includevano:

- La verifica della cooperazione in tempo reale tra i sottosistemi (video, luci, audio, sensori di input);
- L'analisi della stabilità della rete locale e del broker MQTT sotto carico, con traffico simultaneo generato da più client;
- La valutazione della sincronizzazione degli eventi su più stazioni operative in parallelo.

Test delle modalità globali: sleep, wake e shutdown

In questa fase sono state testate anche le funzionalità globali per la gestione automatica delle stanze, utili per il controllo dell'intero sistema in modo centralizzato. In particolare, sono stati utilizzati i seguenti comandi MQTT:

```
mosquitto_pub -h 10.42.0.1 -t smartroom/pi1/cmd -m sleep
mosquitto_pub -h 10.42.0.1 -t smartroom/pi1/cmd -m wake
mosquitto_pub -h 10.42.0.1 -t smartroom/shutdown -m '{ "command":
  "shutdown" }'
```

Questi comandi sono stati utilizzati per verificare:

- **Sleep:** ogni stazione video ha chiuso l'interfaccia grafica JavaFX e spento lo schermo, le stazioni audio hanno interrotto la riproduzione e le plafoniere si sono spente.
- **Wake:** il sistema ha correttamente riavviato le applicazioni locali, riattivato lo schermo, riacceso le plafoniere e ripristinato l'audio della stanza.
- **Shutdown:** tutte le stazioni hanno eseguito lo spegnimento controllato, chiudendo le applicazioni e arrestando il Raspberry Pi.

Conclusioni

Il progetto descritto in questa tesi ha rappresentato una sfida concreta, in cui teoria e pratica sono state necessarie per dare vita a un sistema realmente funzionante in un contesto operativo reale. Il progetto sviluppato ha rappresentato non solo una sfida tecnica, ma anche un'opportunità formativa particolarmente stimolante. Sapere che il sistema realizzato sarà effettivamente utilizzato in un contesto reale ha reso l'intero percorso ancora più motivante e coinvolgente. Esso mi ha permesso di consolidare competenze già acquisite e di approfondire nuovi ambiti, tra cui la progettazione di sistemi embedded, l'integrazione di dispositivi IoT, la comunicazione asincrona tramite MQTT e la programmazione distribuita. Uno degli aspetti più significativi è stato il confronto con vincoli reali, sia fisici che operativi, propri dell'ambiente museale. Il sistema ha dovuto adattarsi a spazi già arredati, a dispositivi elettrici preesistenti e alle esigenze di utilizzo quotidiano da parte di operatori non tecnici. Questo ha richiesto soluzioni robuste, flessibili e il più possibile autonome.

La modularità dell'architettura e la chiarezza del modello a topic MQTT si sono dimostrate ottimali, rendendo il sistema facilmente manutenibile e scalabile. Anche dal punto di vista dell'esperienza utente, i test condotti hanno confermato la reattività e la coerenza del comportamento delle stazioni, contribuendo a un'esperienza guidata fluida e coinvolgente.

Tuttavia, il lavoro ha anche messo in luce alcuni limiti. La mancanza di un'interfaccia di controllo centralizzata ha reso alcune operazioni più macchinose del necessario, specialmente in gestione manuale o in fase di test.

In questo senso, il sistema si è dimostrato solido ma con margini di miglioramento soprattutto sul piano del controllo.

Una direzione promettente consiste nello sviluppo di un pannello di gestione locale da installare fisicamente all'interno della sala. Tale pannello, dotato di interfaccia utente semplice e intuitiva, potrebbe essere connesso alla rete LAN del sistema per comunicare direttamente con il coordinatore centrale o con i dispositivi presenti nella rete.

Le funzionalità del pannello potrebbero includere la visualizzazione dello stato dei dispositivi nella stanza, il controllo delle singole componenti (come l'accensione o lo spegnimento di una specifica unità Shelly senza la necessità

di attivare l'intera stazione video), e l'invio di comandi MQTT per mettere la stanza in modalità standby, riattivarla all'inizio della giornata o per spegnerla completamente.

Un'ulteriore possibile evoluzione del sistema riguarda l'infrastruttura di rete. Attualmente, la connessione tra i dispositivi è garantita dal coordinatore centrale che funge da access point, mettendo a disposizione una rete LAN interna. Sebbene questa soluzione sia funzionale nel contesto attuale, va considerato che il Raspberry Pi non nasce come dispositivo progettato specificamente per fornire servizi di rete avanzati. Di conseguenza, la rete creata dal coordinatore potrebbe non garantire la stessa stabilità, copertura e capacità di gestione tipiche di un router dedicato. In prospettiva, l'adozione di un vero e proprio router per la creazione di una rete Wi-Fi privata autonoma permetterebbe di migliorare la robustezza e l'affidabilità delle comunicazioni, facilitare l'integrazione di nuovi dispositivi e separare in modo più netto i ruoli logici all'interno dell'architettura. Questa modifica renderebbe il sistema ancora più scalabile e resiliente, in vista di possibili sviluppi futuri.

In conclusione, il progetto rappresenta un esempio concreto di trasformazione digitale applicata a un contesto museale, dimostrando come l'adozione di tecnologie smart possa migliorare l'esperienza di visita, semplificare la gestione tecnica e aprire la strada a nuove modalità di fruizione del patrimonio culturale. Le possibilità di estensione rendono il sistema adatto a evolversi nel tempo, mantenendo flessibilità, efficienza e scalabilità come caratteristiche fondamentali.

Bibliografia

- [1] Ala Al-Fuqaha, Mohsen Guizani, Mehdi Mohammadi, Mayan Aledhari, and Moussa Ayyash. Internet of things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, 2015.
- [2] Luigi Atzori, Antonio Iera, and Giacomo Morabito. The internet of things: A survey. *Computer networks*, 2010.
- [3] Carles Gómez, Josep Paradells, Joaquim Oller, and David Roca. Internet of things for enabling smart environments: A technology-centric perspective. *Journal of Ambient Intelligence and Smart Environments*, 2018.
- [4] Somayya Madakam, Ramani Ramaswamy, and Siddharth Tripathi. Internet of things (iot): A literature review. *Journal of Computer and Communications*, 2015.
- [5] Petros Spachos and Konstantinos N. Plataniotis. Ble beacons for indoor positioning in museums. *IEEE Systems Journal*, 2020.